AN AI BASED SECURITY AUDITING FRAMEWORK USING ZEEK CONNECTION LOGS AND UNSUPERVISED MACHINE LEARNING

Abstract

Network security auditing has become increasingly challenging due to the scale and complexity of modern network traffic. Traditional signature-based intrusion detection systems fail to identify zero-day and unknown attack patterns, making unsupervised machine learning approaches essential. This paper presents the design and implementation of an AI-based security auditing tool that captures network traffic using Zeek, processes the generated connection logs (conn.log), and applies three unsupervised anomaly detection models — Isolation Forest (IF), Local Outlier Factor (LOF), and One-Class Support Vector Machine (OCSVM) — to detect abnormal network behaviour. The ML model results were indexed into Elasticsearch and visualized through interactive Kibana dashboards. A total of 263,992 Zeek connection log records were analysed. LOF detected the highest anomaly rate at 10.15%, followed by Isolation Forest at 9.96% and One-Class SVM at 9.45%, with 78,052 total anomalies detected across all three models. The results were visualized across two professional Kibana dashboards containing 18 panels. The system demonstrates a practical, end-to-end offline security auditing framework suitable for network forensics and audit reporting.

Keywords: Zeek, Connection Log Analysis, Isolation Forest, Local Outlier Factor, One-Class SVM, Anomaly Detection, ELK Stack, Kibana, Network Security Auditing, Unsupervised Machine Learning.

1. Introduction

Network security monitoring is a critical function in any modern IT infrastructure. The rapid increase in network traffic volume, combined with the sophistication of modern cyber attacks, has made manual inspection of network activity practically infeasible. Traditional intrusion detection systems (IDS) rely primarily on signature-based or rule-based methods, which are effective only against known and previously catalogued threats. These systems completely fail to detect zero-day

attacks — novel intrusion techniques that have no existing signature — leaving organizations exposed to significant security risks.

Unsupervised machine learning approaches offer a powerful alternative. As demonstrated by Nassif et al. [9] in their systematic review of machine learning for anomaly detection, unsupervised methods do not require labeled attack data and are capable of detecting deviations from normal behavior purely by learning the statistical structure of regular network traffic. This makes them especially well-suited for security auditing environments where labeled attack samples are not available during model training.

Zeek (formerly known as Bro), introduced by Paxson [1], is a widely adopted open-source network security monitoring framework that transforms raw network packets into high-level, structured log files. Among these, the conn.log file encodes rich behavioral metadata for every observed network session — including source and destination IP addresses, port numbers, protocol type, bytes transferred, connection state, and session duration — making it an ideal input for machine learning-based anomaly detection. Moomtaheen et al. [2] confirmed that Zeek connection log data provides sufficient feature richness for effective unsupervised anomaly detection using Isolation Forest, reinforcing Zeek's suitability as the data source in this project.

This project presents a complete end-to-end security auditing tool built entirely around Zeek's conn.log. The pipeline flows as: **Zeek** for network traffic capture and structured connection log generation → **Google Colab** for ML model training and inference → **Elasticsearch** for data indexing → **Kibana** for interactive dashboard visualization. Three unsupervised models are applied and compared — Isolation Forest [4], Local Outlier Factor [5], and One-Class SVM [6] — without requiring any labeled attack data, making the system suitable for zero-day and unknown threat detection in offline audit scenarios. The comparative multi-model approach follows the evaluation methodology established by Andrews et al. [3] and validated by Aldea et al. [8] for unsupervised network anomaly detection.

The rest of this paper is organized as follows: Section 2 presents the literature survey, Section 3 describes the proposed system architecture and methodology, Section 4 presents the workflow, Section 5 covers the algorithms, Section 6

describes the implementation, Section 7 presents results and Kibana dashboard analysis, Section 8 concludes the paper, followed by references.

2. Literature Survey

2.1 Zeek for Network Traffic Monitoring

Paxson [1] introduced Bro (now Zeek), a stand-alone system for detecting network intruders in real-time by passively monitoring a network link. Zeek is designed as an event engine that reduces a kernel-filtered network traffic stream into a series of higher-level structured events, making raw packet captures interpretable as semantic log records. Its conn.log format encodes per-connection metadata including timestamps, IP addresses, ports, protocol, service, bytes, duration, and connection state — providing a structured, feature-rich dataset directly suitable for machine learning analysis. Zeek has since become one of the most widely deployed open-source network security monitoring tools in both research and enterprise environments.

2.2 Zeek Connection Logs Combined with Machine Learning

Moomtaheen et al. [2] investigated the use of Extended Isolation Forest applied directly to Zeek connection log data for intrusion detection. Their work demonstrated that Zeek's conn.log records, once preprocessed and normalized, contain sufficient discriminative signal for unsupervised anomaly detection models to identify abnormal network sessions. This directly validates the data source and model selection approach adopted in the present project, where the same Zeek conn.log format is used as input to three unsupervised models including Isolation Forest.

2.3 Comparing Machine Learning Techniques on Zeek Logs

Andrews et al. [3] conducted a systematic comparison of multiple machine learning techniques applied to Zeek log data from enterprise network environments. Their work established a benchmark methodology for evaluating unsupervised ML models on Zeek-derived features, demonstrating that different algorithms exhibit different sensitivity profiles across traffic types. This

comparative philosophy directly motivates the parallel application of three models — IF, LOF, and OCSVM — in the present project, enabling a fair and structured evaluation of their anomaly detection behavior on the same Zeek conn.log dataset.

2.4 Isolation Forest for Anomaly Detection

Liu et al. [4] proposed Isolation Forest, an ensemble anomaly detection algorithm that isolates anomalies by recursively partitioning the feature space using random cuts. The algorithm exploits the observation that anomalies are few in number and structurally different from normal points, meaning they require fewer random partitions to be isolated. Isolation Forest has since become one of the most widely applied unsupervised anomaly detection algorithms due to its linear time complexity and effectiveness on high-dimensional data — properties particularly valuable when analyzing large-scale network connection logs.

2.5 Local Outlier Factor for Density-Based Anomaly Detection

Breunig et al. [5] introduced Local Outlier Factor, a density-based algorithm that computes a local anomaly score for each point by comparing its local reachability density to those of its k nearest neighbors. Unlike global statistical methods, LOF captures local cluster structure, enabling it to identify anomalies that are only unusual relative to their immediate neighborhood — a property particularly valuable in network traffic data where normal behavior varies significantly across subnets, protocols, and time periods.

2.6 One-Class SVM for Boundary-Based Anomaly Detection

Schölkopf et al. [6] introduced One-Class SVM, which learns a kernel-based boundary enclosing the normal training data in a high-dimensional feature space. At inference, data points falling outside this learned boundary are classified as anomalies. OCSVM's use of the RBF kernel makes it effective for non-linearly separable data distributions, which are commonly observed in real-world network connection logs where normal and anomalous traffic do not separate linearly in the raw feature space.

2.7 Comparing IF, LOF, and OCSVM for Network Security

Sarwar et al. [7] conducted a direct comparison of Isolation Forest and One-Class SVM for IoT network security, demonstrating that both models achieve competitive anomaly detection performance but differ in their sensitivity to boundary conditions and contamination rates. Aldea et al. [8] extended this comparison to include all three models — IF, LOF, and OCSVM — in a critical infrastructure security context, finding that no single model dominates across all traffic categories and that a multi-model comparative approach produces the most reliable security assessment. This finding directly motivates the three-model parallel deployment in the present project.

2.8 Systematic Review of ML Anomaly Detection Methods

Nassif et al. [9] conducted a systematic review of machine learning methods for anomaly detection covering over 300 papers. Their review confirms that Isolation Forest, LOF, and One-Class SVM are among the most frequently applied and best-performing unsupervised anomaly detection algorithms across cybersecurity domains. The review also establishes that unsupervised methods are most appropriate when labeled training data is unavailable — precisely the condition under which the present system operates.

2.9 ELK Stack for Security Log Visualization

Srivastava and Haritha [10] demonstrated the use of the Elastic Stack (Elasticsearch + Kibana) as a Security Information and Event Management (SIEM) platform for network log analysis and threat detection. Their work confirms that Kibana dashboards provide security analysts with actionable, interpretable visualizations of large-scale log data without requiring deep technical expertise. This directly supports the dashboard visualization layer in the present project, where 18 Kibana panels across two dashboards present ML anomaly detection results to the analyst in an interactive, explorable format.

2.10 Research Gap

Despite strong individual bodies of work on Zeek log analysis [1][2][3], unsupervised anomaly detection algorithms [4][5][6][7][8][9], and ELK-based security visualization [10], no existing work integrates all three components —

Zeek connection log generation, three-model unsupervised ML comparison, and full Kibana dashboard reporting — into a single offline, label-free security auditing pipeline. This project addresses that specific gap.

3. Proposed Methodology

3.1 System Architecture Overview

The proposed system operates as a five-stage pipeline specifically designed around Zeek's conn.log as the sole data source:

Stage 1: Network traffic capture using Zeek 6.0 to generate structured conn.log files from a real local area network [1].

Stage 2: Data preprocessing and feature engineering in Google Colab — missing value handling, categorical encoding, feature selection, and Min-Max normalization.

Stage 3: ML model training and inference using Isolation Forest [4], LOF [5], and OCSVM [6] applied simultaneously to all 263,992 connection log records.

Stage 4: Results export to CSV and bulk indexing into Elasticsearch 8.11.0 across two indices.

Stage 5: Interactive visualization using two Kibana dashboards — Dashboard 1 for raw Zeek traffic analysis (10 panels) and Dashboard 2 for ML anomaly detection model comparison (8 panels).

3.2 Data Collection — Zeek Connection Logs

The dataset was collected from a real local area network. Zeek 6.0 was installed on the host machine and configured to listen on the active network interface in passive mode — it only observed traffic and never injected packets back into the network. Every time a connection finished, Zeek automatically wrote one record to the conn.log file, covering all traffic types: TCP, UDP, and ICMP. By the time the capture ended, the log contained **263,992 records** in total.

The traffic included everyday services such as DNS, HTTP, and SSL. None of it was artificially generated or imported from an external dataset — it was real traffic from a monitored local network, making the anomaly detection results reflect actual normal behavior rather than a lab configuration.

Each conn.log record encodes the following fields:

ts — Time the connection started.

uid — Unique ID for each connection.

id.orig_h — Source IP address.

id.orig_p — Source port number.

id.resp_h — Destination IP address.

id.resp_p — Destination port number.

proto — Protocol used (TCP, UDP, ICMP).

service — Application service detected (DNS, HTTP, SSL).

duration — How long the connection lasted in seconds.

orig_bytes — Bytes sent by the source.

resp_bytes — Bytes sent by the destination.

orig_pkts — Packets sent by the source.

resp_pkts — Packets sent by the destination.

conn_state — How the connection ended (SF, S0, REJ, etc.).

Each of these fields provides meaningful behavioral signal. For example, a connection that transferred a large volume of bytes in a very short duration is inherently suspicious. A connection that never properly closed (indicated by conn_state) may indicate scanning or an incomplete handshake. Together, these fields provide a complete behavioral fingerprint of every network session.

3.3 Data Preprocessing and Feature Engineering

Raw Zeek conn.log files are not directly usable for machine learning because they contain missing values and text-based categorical fields that models cannot process numerically. The following preprocessing pipeline was applied:

Step 1 – Missing Value Handling: Zeek represents undefined or unavailable field values with the '-' symbol. For example, some UDP connections do not have a duration value because UDP has no formal closing handshake. All '-' entries were replaced with 0, as treating absent numeric measurements as zero is the most logically consistent approach.

Step 2 – Categorical Encoding: Text-based fields were converted to numeric values using standard encoding:

- Protocol: TCP = 6, UDP = 17, ICMP = 1 (following IANA standard protocol numbers)
- Service: DNS, HTTP, SSL encoded as integer labels starting from 1
- Connection state: SF, S0, REJ, and other state codes encoded as integer labels

Step 3 – Feature Selection: Eight features were selected as model input:

- Source port, Destination port, Protocol (encoded), Service (encoded), Duration, Bytes sent (orig_bytes), Bytes received (resp_bytes), Connection state (encoded)

Fields such as the timestamp (ts) and connection ID (uid) were dropped because they are unique per record and carry no generalizable statistical pattern useful for anomaly detection.

Step 4 – Variance Check: The Z-score was computed for each feature to verify that no feature had near-zero variance:

$$z = (x - \mu) / \sigma$$

where μ is the mean and σ is the standard deviation. Features with σ close to zero were removed as they carry no discriminative information.

Feature variance was also calculated to confirm each feature was worth retaining:

$$\mathrm{Var}(f_i) = (1/N) \times \Sigma\square\ (f_i\square - \mathrm{mean}(f_i))^2$$

Any feature falling below the acceptable variance threshold was excluded before model training.

Step 5 – Min-Max Normalization: Because features differ vastly in scale — orig_bytes can reach into the thousands while duration is typically less than 1 — Min-Max normalization was applied to scale all features to the range [0, 1]:

$$x' = (x - x_min) / (x_max - x_min)$$

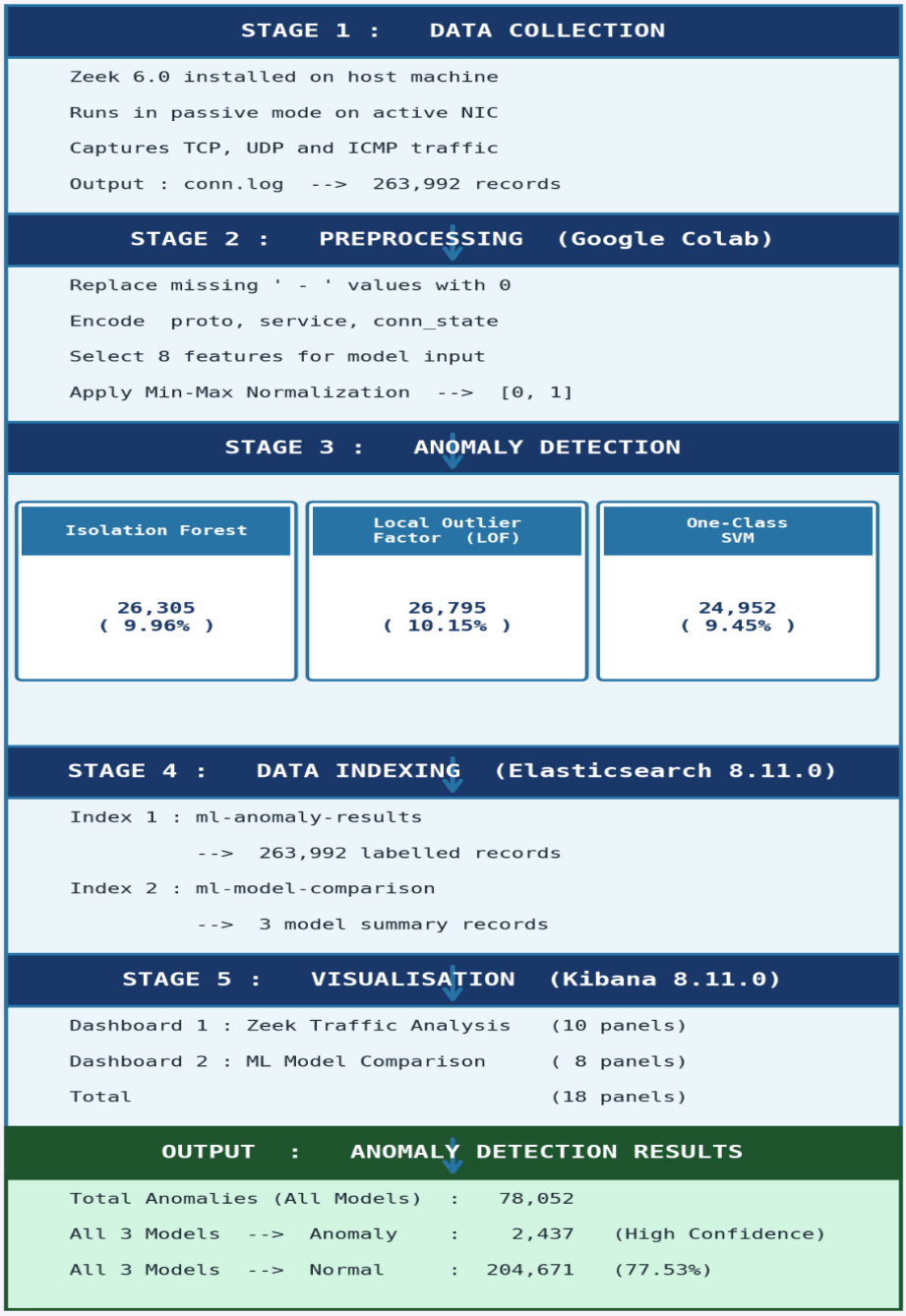
Where x is the original value, x_min and x_max are the minimum and maximum values of that feature across the dataset, and x' is the normalized output. This ensures no single large-valued feature dominates model training.

The Euclidean distance formula used internally by LOF and OCSVM when comparing two connection records p and q across all 8 features is:

$$d(p, q) = \sqrt{(\Sigma_{i=1}^{8}\ (p_i - q_i)^2)}$$

4. Workflow Diagram

The end-to-end pipeline for Zeek connection log anomaly detection follows five sequential stages:



5. Algorithms

5.1 Isolation Forest

Randomly builds decision trees and measures how quickly each record gets isolated. Anomalies are rare and different — they get isolated in fewer splits than normal records.

Anomaly Score:

$s(x, n) = 2^{(-E[h(x)] / c(n))}$

- Score near **1.0** → Anomaly
- Score near **0.5** → Normal
- Score near **0.0** → Deeply normal

Parameters: n_estimators = 200, contamination = 0.1, random_state = 42

Result: 26,305 anomalies (9.96%)

5.2 Local Outlier Factor (LOF)

Compares each record's local density against its k nearest neighbors. A point much less dense than its neighbors is flagged as an anomaly.

LOF Score:

$LOF_k(p) = (\Sigma\ lrd_k(o) / lrd_k(p)) / |N_k(p)|$

- LOF ≈ 1 → Normal (similar density to neighbors)
- LOF >> 1 → Anomaly (much sparser than neighbors)

Parameters: n_neighbors = 20, contamination = 0.1, novelty = True

Result: 26,795 anomalies (10.15%) — Highest

5.3 One-Class SVM (OCSVM)

Learns a boundary around normal traffic using an RBF kernel. Any record falling outside the boundary is classified as an anomaly.

Decision Function:

$f(x) = w \cdot \Phi(x) - \rho$

- $f(x) \geq 0 \rightarrow$ Normal (inside boundary)
- $f(x) < 0 \rightarrow$ Anomaly (outside boundary)

Parameters: kernel = rbf, nu = 0.1, gamma = scale

Result: 24,952 anomalies (9.45%) — Lowest

6. Implementation & Results

```
#
================================================================================
==

# CELL 1: SETUP & DATA LOADING

#
================================================================================
==

# CRITICAL: Run this ENTIRE cell - do not skip the imports at the top

#
================================================================================
==

# IMPORTS FIRST - DO NOT SKIP THESE LINES

import numpy as np

import pandas as pd

import warnings

warnings.filterwarnings('ignore')

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.feature_selection import VarianceThreshold

from sklearn.ensemble import IsolationForest, RandomForestClassifier

from sklearn.svm import OneClassSVM

from sklearn.neighbors import LocalOutlierFactor
```

```
from sklearn.decomposition import PCA

from datetime import datetime

plt.style.use('seaborn-v0_8-darkgrid')

sns.set_palette("husl")

pd.set_option('display.max_columns', None)

print("✓ All libraries imported")

print(f"Timestamp: {datetime.now()}")
```

```
✓ All libraries imported

Timestamp: 2026-02-26 11:42:58.404864
```

```
#
==========================================================================
==

# DATA LOADING FUNCTION

#
==========================================================================
==

def parse_zeek_log(filepath):

    """Parse Zeek/Bro conn.log files."""

    separator = '\t'

    columns = []

    with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:

        for line in f:

            if line.startswith('#separator'):

                separator = '\t' if '\\x09' in line else ' '

            elif line.startswith('#fields'):

                columns = line.strip().split('\t')[1:]

            elif not line.startswith('#'):

                break

    print(f"Detected {len(columns)} columns")
```

```
    df = pd.read_csv(
        filepath,
        sep=separator,
        names=columns,
        comment='#',
        skip_blank_lines=True,
        low_memory=False,
        on_bad_lines='skip'
    )
    return df

#
==========================================================================
==

# UPLOAD FILE

#
==========================================================================
==

from google.colab import files

print("\n" + "="*80)

print("Please upload your conn.log file:")

print("="*80)

uploaded = files.upload()

filename = list(uploaded.keys())[0]

#
==========================================================================
==

# PARSE THE FILE

#
==========================================================================
==

print(f"\nParsing: {filename}")
```

```
df_conn = parse_zeek_log(filename)

print(f"\n✓ Loaded: {df_conn.shape[0]:,} rows × {df_conn.shape[1]}
columns")

print(f"  Memory: {df_conn.memory_usage(deep=True).sum() / 1024**2:.2f}
MB")

#
==========================================================================
==

# HANDLE ZEEK MISSING VALUES

#
==========================================================================
==

df_conn.replace(['-', '(empty)', 'empty'], np.nan, inplace=True)

print("✓ Zeek missing value indicators replaced with NaN")

==========================================================================
======

Please upload your conn.log file:

==========================================================================
======

conn.log(n/a) - 152560913 bytes, last modified: 1/1/2026 - 100% done

Saving conn.log to conn (2).log

Parsing: conn (2).log

Detected 21 columns

✓ Loaded: 1,319,960 rows × 21 columns

  Memory: 961.60 MB

✓ Zeek missing value indicators replaced with NaN

#
==========================================================================
==

# DATA TYPE CONVERSION

#
==========================================================================
==
```

```
print("\nConverting data types...")

numeric_candidates = [

    'duration', 'orig_bytes', 'resp_bytes', 'missed_bytes',

    'orig_pkts', 'orig_ip_bytes', 'resp_pkts', 'resp_ip_bytes'

]

for col in numeric_candidates:

    if col in df_conn.columns:

        df_conn[col] = pd.to_numeric(df_conn[col], errors='coerce')

if 'ts' in df_conn.columns:

    df_conn['ts'] = pd.to_numeric(df_conn['ts'], errors='coerce')

    df_conn['timestamp'] = pd.to_datetime(df_conn['ts'], unit='s',
errors='coerce')

print("✓ Data types converted")

#
==========================================================================
==

# DISPLAY SAMPLE

#
==========================================================================
==

print(f"\nDataFrame ready: {df_conn.shape}")

print("\nFirst 5 rows:")

display(df_conn.head())

print("\n" + "="*80)

print("✓ CELL 1 COMPLETE - Run Cell 2 next")

print("="*80)
```

```
Converting data types...
✓ Data types converted

DataFrame ready: (1319960, 22)

First 5 rows:
```

	ts	uid	id.orig_h	id.orig_p	id.resp_h	id.resp_p	proto	service	duration	orig_bytes	resp_bytes	conn_state	local_orig	local_resp	missed_bytes	history
0	1.556714e+09	Chmpt162NcHAq0YCb	192.168.61.20	50885	104.81.135.169	80	tcp	http	0.373957	213.0	4657.0	SF	NaN	NaN	0	ShADadfF
1	1.556714e+09	CP1SQx2NHC8s9Z0CBi	192.168.61.20	50883	64.4.54.165	443	tcp	NaN	0.058941	0.0	0.0	REJ	NaN	NaN	0	FRr
2	1.556714e+09	CuBvgmZmRctyexTO7	192.168.50.25	50074	8.8.8.8	53	udp	dns	0.036838	63.0	168.0	SF	NaN	NaN	0	Dd
3	1.556714e+09	CBRo9C4IGZ28ds6kSj	192.168.50.25	49869	8.8.8.8	53	udp	dns	0.014214	40.0	115.0	SF	NaN	NaN	0	Dd
4	1.556714e+09	CAnMKM3FVungeQbAci	192.168.50.25	51367	8.8.8.8	53	udp	dns	0.113166	71.0	133.0	SF	NaN	NaN	0	Dd

```
================================================================================
✓ CELL 1 COMPLETE - Run Cell 2 next
================================================================================
```

```
# ================================================================================
# CELL 2: EXPLORATORY DATA ANALYSIS
# ================================================================================
# Run this cell after Cell 1 completes successfully
# ================================================================================
# Check prerequisite
assert 'df_conn' in dir() and df_conn is not None, \
    "❌ Run Cell 1 first to load df_conn!"
print("="*80)
print("EXPLORATORY DATA ANALYSIS")
print("="*80)
# ================================================================================
# DATASET OVERVIEW
# ================================================================================
print("\n--- DATASET OVERVIEW ---")
```

```
print(f"Shape: {df_conn.shape}")

print(f"\nColumns ({len(df_conn.columns)}):")

print(df_conn.columns.tolist())

print(f"\nData types:")

print(df_conn.dtypes)

print(f"\nRandom sample (10 rows):")

display(df_conn.sample(min(10, len(df_conn))))

print("\nStatistical summary:")

display(df_conn.describe(include='all').T)
```

```
================================================================================
EXPLORATORY DATA ANALYSIS
================================================================================

--- DATASET OVERVIEW ---
Shape: (1319960, 22)

Columns (22):
['ts', 'uid', 'id.orig_h', 'id.orig_p', 'id.resp_h', 'id.resp_p', 'proto', 'service', 'duration', 'orig_bytes', 'resp_bytes', 'conn_state', 'local_orig', 'local_resp', 'missed_bytes', 'history',

Data types:
ts                     float64
uid                     object
id.orig_h               object
id.orig_p                int64
id.resp_h               object
id.resp_p                int64
proto                   object
service                 object
duration               float64
orig_bytes             float64
resp_bytes             float64
conn_state              object
local_orig             float64
local_resp             float64
missed_bytes             int64
history                 object
orig_pkts                int64
orig_ip_bytes            int64
resp_pkts                int64
resp_ip_bytes            int64
tunnel_parents         float64
timestamp       datetime64[ns]
dtype: object

Random sample (10 rows):
```

	ts	uid	id.orig_h	id.orig_p	id.resp_h	id.resp_p	proto	service	duration	orig_bytes	resp_bytes	conn_state	local_orig	local_resp	missed_bytes
158874	1.556817e+09	COjqeq1FB9S9qDtuQe	192.168.10.43	20390	179.5.108.78	22	tcp	NaN	NaN	NaN	NaN	S0	NaN	NaN	0
355414	1.556817e+09	Clv1Ua36Z5OAAEPk9e	192.168.10.43	54650	111.20.44.99	22	tcp	NaN	NaN	NaN	NaN	S0	NaN	NaN	0
207369	1.556817e+09	C540V93nFlnlWsP2hc	192.168.10.43	19622	210.33.105.224	23	tcp	NaN	NaN	NaN	NaN	S0	NaN	NaN	0
1267773	1.556881e+09	CJNxiu4xeGieOcFNsh	192.168.10.43	58904	192.168.60.4	80	tcp	NaN	NaN	NaN	NaN	OTH	NaN	NaN	0
431157	1.556818e+09	CegJfP3PBKpNpTGy27	192.168.10.43	19622	94.122.247.44	23	tcp	NaN	NaN	NaN	NaN	S0	NaN	NaN	0
1131265	1.556871e+09	C6rOJq1JLzs01Pwsr2	192.168.10.43	58234	192.168.61.50	1700	tcp	NaN	0.000275	0.0	0.0	REJ	NaN	NaN	0

451157	1.556818e+09	CegJIP3PBKpNpTGy27	192.168.10.43	19622	94.122.247.44	23	tcp	NaN	NaN	NaN	NaN	S0	NaN	NaN	0
1131265	1.556871e+09	C6rOJq1JLzs01Pwsr2	192.168.10.43	58234	192.168.61.50	1700	tcp	NaN	0.000275	0.0	0.0	REJ	NaN	NaN	0
1151130	1.556874e+09	C07XWy1IkrGbWOW771	192.168.10.50	5519	192.168.49.96	443	tcp	NaN	NaN	NaN	NaN	S0	NaN	NaN	0
1053213	1.556865e+09	CrDwSRLDRVymE0hWb	192.168.10.43	46648	104.211.160.15	80	tcp	http	67.353978	791.0	18642.0	SF	NaN	NaN	0
48538	1.556737e+09	CGNZpJ2FVmtDnEtpcb	192.168.10.43	57900	192.168.10.50	1271	tcp	NaN	NaN	NaN	NaN	S0	NaN	NaN	0
1057505	1.556862e+09	CfvQ2s2lyLfFfK8Dsh	185.61.150.201	49846	10.200.200.80	22	tcp	NaN	31.369856	0.0	0.0	S1	NaN	NaN	0

Statistical summary:

	count	unique	top	freq	mean	min	25%	50%	75%	max	std
ts	1319960.0	NaN	NaN	NaN	1556833258.028628	1556714094.311009	1556817326.145166	1556832182.740784	1556859673.820172	1556896704.968338	35736.499905
uid	1319960	1319960	CDX93h4UPJQ6TOprcf	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
id.orig_h	1319960	9746	192.168.10.43	894218	NaN	NaN	NaN	NaN	NaN	NaN	NaN
id.orig_p	1319960.0	NaN	NaN	NaN	35724.487239	3.0	19622.0	38736.0	51816.0	65535.0	18246.149263
id.resp_h	1319960	342166	192.168.10.50	95853	NaN	NaN	NaN	NaN	NaN	NaN	NaN
id.resp_p	1319960.0	NaN	NaN	NaN	3055.317973	0.0	23.0	80.0	1688.0	65389.0	8863.808119
proto	1319960	3	tcp	1196662	NaN	NaN	NaN	NaN	NaN	NaN	NaN
service	171620	11	http	80580	NaN	NaN	NaN	NaN	NaN	NaN	NaN
duration	611873.0	NaN	NaN	NaN	13.096743	0.000008	0.000413	0.322581	7.504488	182617.726542	377.106323
orig_bytes	611873.0	NaN	NaN	NaN	511272.117992	0.0	0.0	0.0	73.0	5331163021.0	33201183.044904
resp_bytes	611873.0	NaN	NaN	NaN	8463.676057	0.0	0.0	0.0	135.0	438813365.0	1001559.55757
conn_state	1319960	13	S0	647224	NaN	NaN	NaN	NaN	NaN	NaN	NaN
local_orig	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
local_resp	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
missed_bytes	1319960.0	NaN	NaN	NaN	0.000464	0.0	0.0	0.0	0.0	176.0	0.274325
history	1234794	1338	S	642702	NaN	NaN	NaN	NaN	NaN	NaN	NaN
orig_pkts	1319960.0	NaN	NaN	NaN	4.93912	0.0	1.0	1.0	2.0	356903.0	428.490421
orig_ip_bytes	1319960.0	NaN	NaN	NaN	636.964056	0.0	40.0	44.0	74.0	52091185.0	85833.868638
resp_pkts	1319960.0	NaN	NaN	NaN	5.116841	0.0	0.0	0.0	1.0	356903.0	634.347099
resp_ip_bytes	1319960.0	NaN	NaN	NaN	4231.679507	0.0	0.0	0.0	40.0	472415537.0	724670.732655
tunnel_parents	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
timestamp	1319960	NaN	NaN	NaN	2019-05-02 21:40:58.028627712	2019-05-01 12:34:54.311008930	2019-05-02 17:15:26.145166336	2019-05-02 21:23:02.740784128	2019-05-03 05:01:13.820171776	2019-05-03 15:18:24.968338013	NaN

```
# ================================================================================
# MISSING VALUES ANALYSIS
# ================================================================================
print("\n--- MISSING VALUES ANALYSIS ---")
missing_stats = pd.DataFrame({
    'Column': df_conn.columns,
    'Missing_Count': df_conn.isnull().sum(),
    'Missing_Percentage': (df_conn.isnull().sum() / len(df_conn) * 100).round(2),
    'Data_Type': df_conn.dtypes
```

```
})

missing_stats = missing_stats[missing_stats['Missing_Count'] >
0].sort_values(

    'Missing_Percentage', ascending=False

)

if not missing_stats.empty:

    print("\nMissing Values Summary:")

    display(missing_stats)

    # Visualizations

    fig, axes = plt.subplots(1, 2, figsize=(16, 6))

    # Heatmap (sample)

    sample_df = df_conn.sample(min(10000, len(df_conn)))

    sns.heatmap(sample_df.isnull(), cbar=True, cmap='viridis',

                ax=axes[0], yticklabels=False)

    axes[0].set_title('Missing Data Heatmap (10k sample)', fontsize=14,
weight='bold')

    # Bar chart

    missing_stats.head(15).plot(

        x='Column', y='Missing_Percentage',

        kind='barh', ax=axes[1], color='coral', legend=False

    )

    axes[1].set_title('Missing Data % by Column (Top 15)', fontsize=14,
weight='bold')

    axes[1].set_xlabel('Missing Percentage (%)')

    axes[1].set_ylabel('Column')

    plt.tight_layout()

    plt.show()

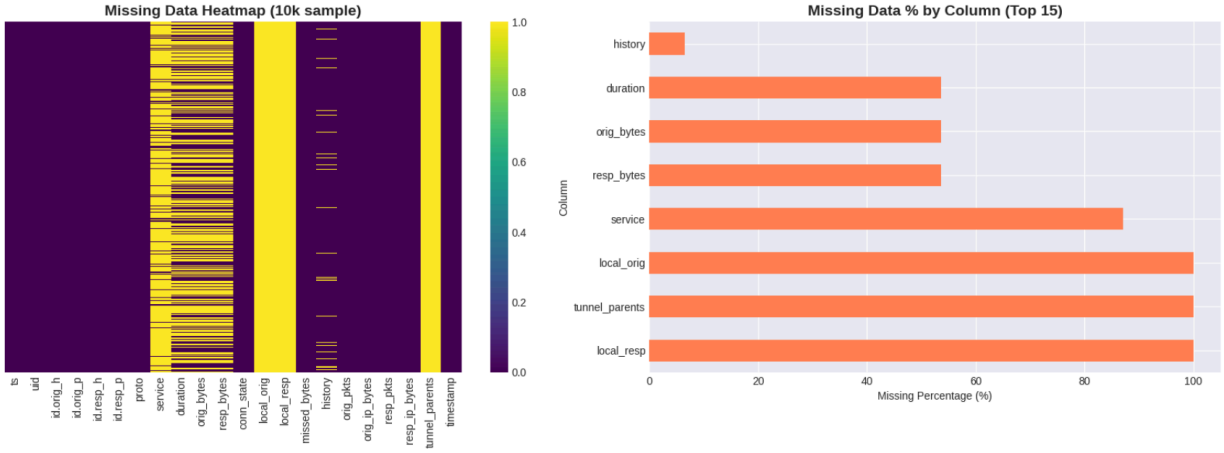
else:

    print("\n✓ No missing values found")
```

```
--- MISSING VALUES ANALYSIS ---

Missing Values Summary:
```

	Column	Missing_Count	Missing_Percentage	Data_Type
local_resp	local_resp	1319960	100.00	float64
tunnel_parents	tunnel_parents	1319960	100.00	float64
local_orig	local_orig	1319960	100.00	float64
service	service	1148340	87.00	object
resp_bytes	resp_bytes	708087	53.64	float64
orig_bytes	orig_bytes	708087	53.64	float64
duration	duration	708087	53.64	float64
history	history	85166	6.45	object




```
# ================================================================================
# NUMERICAL FEATURES DISTRIBUTION
# ================================================================================

print("\n--- NUMERICAL FEATURES DISTRIBUTION ---")

numerical_cols = [c for c in
df_conn.select_dtypes(include=np.number).columns
                  if c not in ['ts']]
```

```
print(f"Numerical columns ({len(numerical_cols)}): {numerical_cols}")
key_numerical = [c for c in ['duration', 'orig_bytes', 'resp_bytes',
                             'orig_pkts', 'resp_pkts', 'orig_ip_bytes',
'resp_ip_bytes']
                 if c in df_conn.columns]
if key_numerical:
    plot_features = key_numerical[:4]
    fig, axes = plt.subplots(len(plot_features), 2,
                             figsize=(16, 4 * len(plot_features)))
    if len(plot_features) == 1:
        axes = axes.reshape(1, -1)
    for idx, col in enumerate(plot_features):
        data = df_conn[col].dropna()
        if len(data) == 0:
            continue
        # Histogram
        axes[idx, 0].hist(data, bins=50, color='skyblue',
                          edgecolor='black', alpha=0.7)
        axes[idx, 0].set_title(f'{col} - Distribution', weight='bold')
        axes[idx, 0].set_xlabel(col)
        axes[idx, 0].set_ylabel('Frequency')
        mean_val = data.mean()
        median_val = data.median()
        axes[idx, 0].axvline(mean_val, color='red', linestyle='--',
                             label=f'Mean: {mean_val:.2f}')
        axes[idx, 0].axvline(median_val, color='green', linestyle='--',
                             label=f'Median: {median_val:.2f}')
        axes[idx, 0].legend()
```

```
        axes[idx, 0].set_yscale('log')

        # Boxplot

        axes[idx, 1].boxplot(data, vert=False)

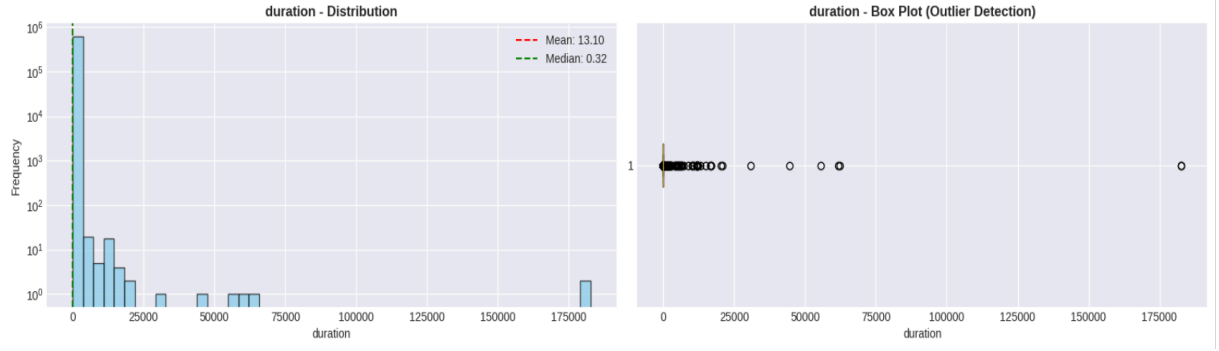
        axes[idx, 1].set_title(f'{col} - Box Plot (Outlier Detection)',
weight='bold')

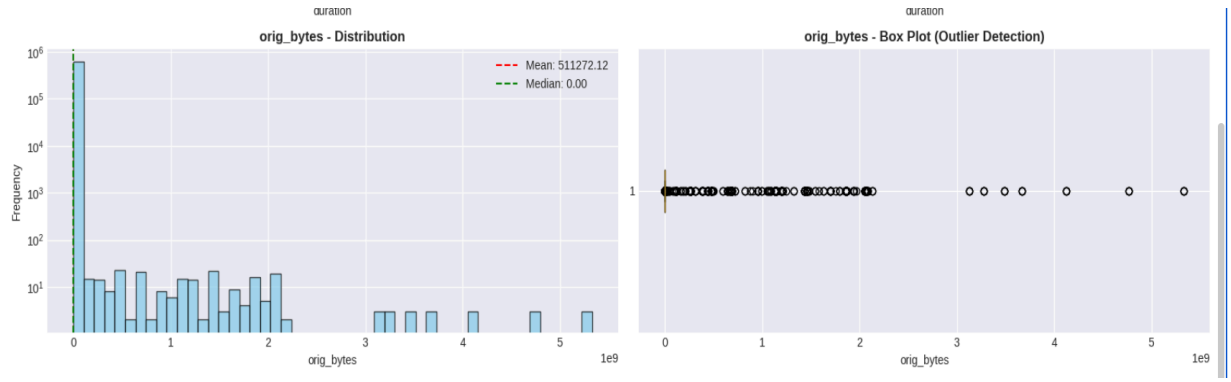
        axes[idx, 1].set_xlabel(col)

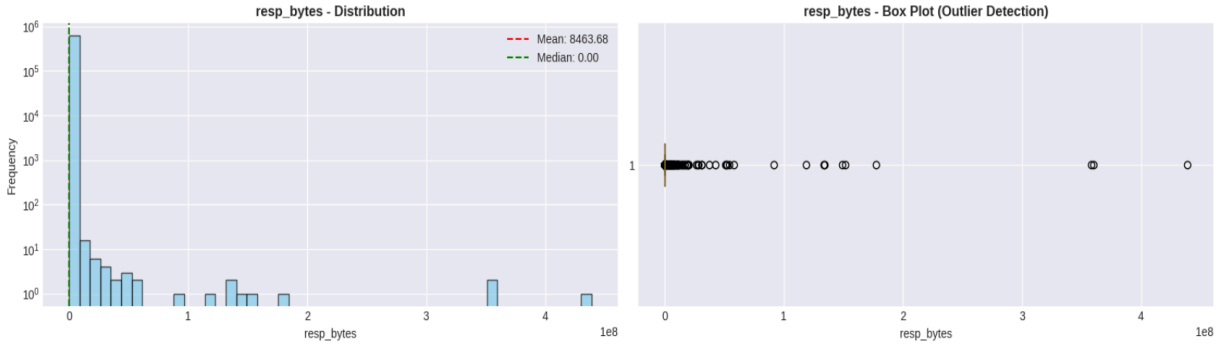
    plt.tight_layout()

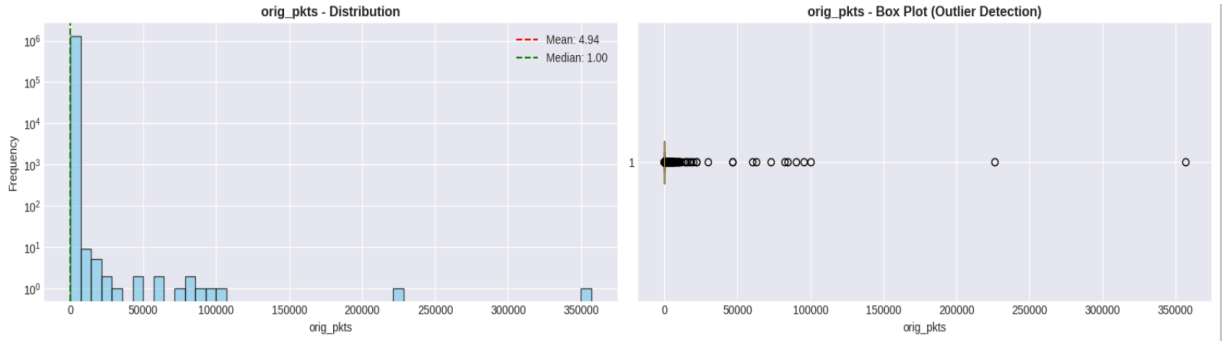
    plt.show()
```

```
...
--- NUMERICAL FEATURES DISTRIBUTION ---
Numerical columns (13): ['id.orig_p', 'id.resp_p', 'duration', 'orig_bytes', 'resp_bytes', 'local_orig', 'local_resp', 'missed_bytes', 'orig_pkts', 'orig_ip_bytes', 'resp_pkts', 'resp_ip_bytes',
```










```
#
==========================================================================
==

# CATEGORICAL FEATURES ANALYSIS

#
==========================================================================
==

print("\n--- CATEGORICAL FEATURES ANALYSIS ---")

categorical_cols =
df_conn.select_dtypes(include='object').columns.tolist()

print(f"Categorical columns ({len(categorical_cols)}):
{categorical_cols}")

key_categorical = [c for c in ['proto', 'service', 'conn_state',
'history']

                   if c in df_conn.columns]

for col in key_categorical:

    print(f"\n{col} - Value Counts:")

    value_counts = df_conn[col].value_counts()

    print(value_counts.head(15))

    # Visualization

    fig, ax = plt.subplots(figsize=(12, 5))

    value_counts.head(15).plot(kind='bar', color='teal', ax=ax)

    ax.set_title(f'{col} - Distribution (Top 15)', fontsize=14,
weight='bold')

    ax.set_xlabel(col)
```

```
    ax.set_ylabel('Count')

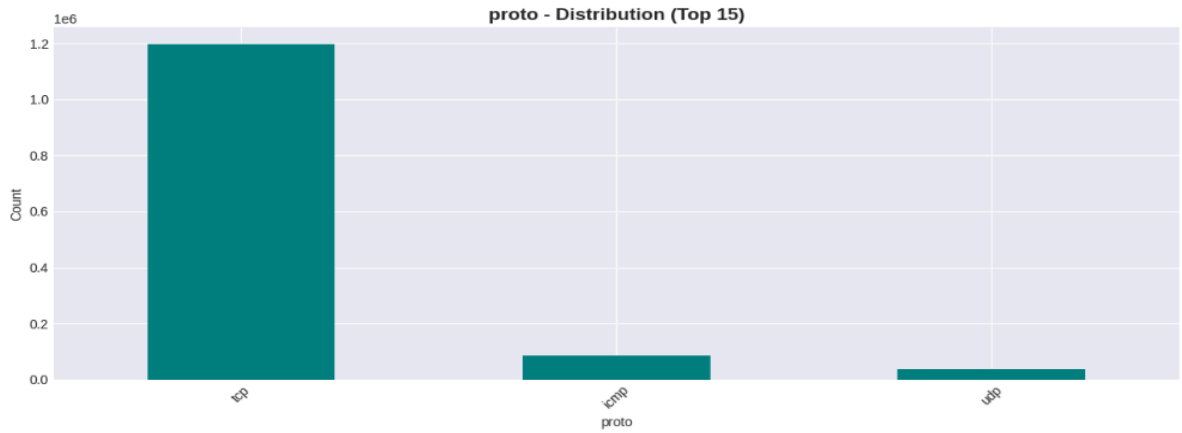
    ax.tick_params(axis='x', rotation=45)

    plt.tight_layout()

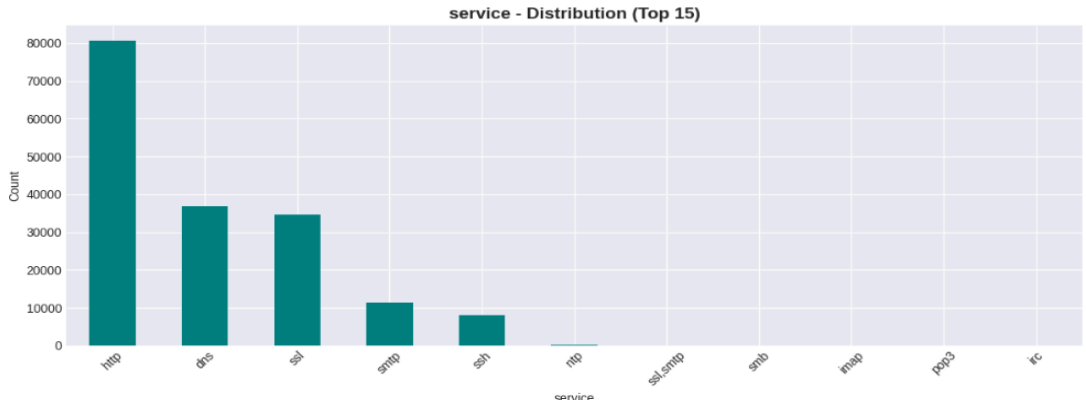
    plt.show()
```

```
--- CATEGORICAL FEATURES ANALYSIS ---
Categorical columns (7): ['uid', 'id.orig_h', 'id.resp_h', 'proto', 'service', 'conn_state', 'history']

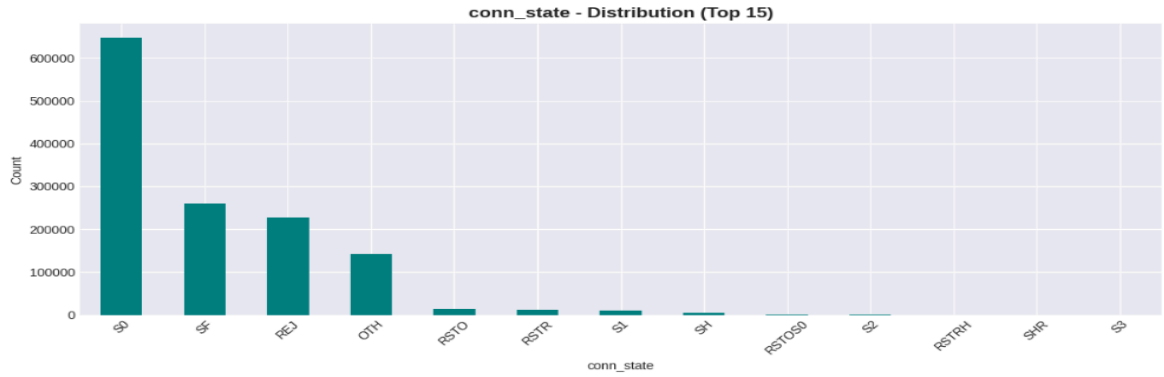
proto - Value Counts:
proto
tcp     1196662
icmp      85085
udp       38213
Name: count, dtype: int64
```



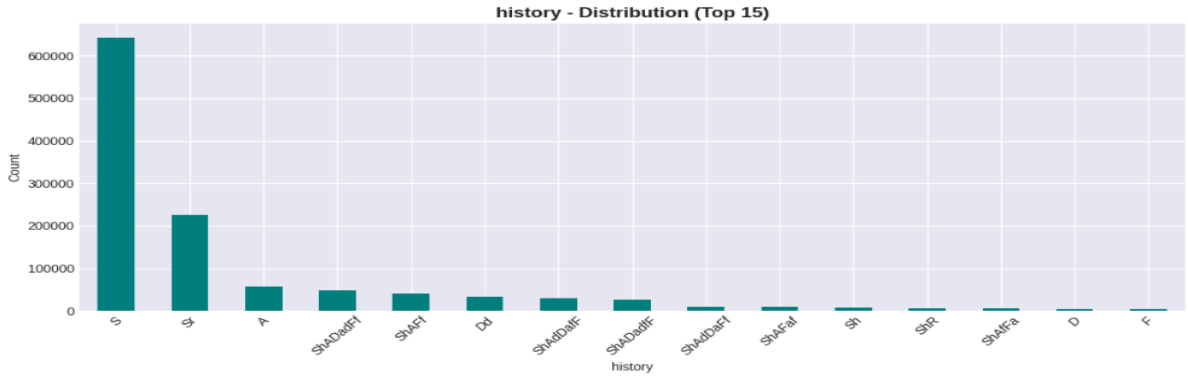

```
service - Value Counts:
service
http        80580
dns         36871
ssl         34689
smtp        11296
ssh          7892
ntp           240
ssl,smtp       32
smb             9
imap            4
pop3            4
irc             3
Name: count, dtype: int64
```



```
conn_state - Value Counts:
conn_state
S0        647224
SF        259992
REJ       227555
OTH       142044
RSTO       13639
RSTR       12080
S1          9448
SH          5452
RSTOS0      1771
S2           257
RSTRH        186
SHR          181
S3           131
Name: count, dtype: int64
```




```
history - Value Counts:
history
S           642702
Sr          226310
A            56759
ShADadFf     48072
ShAFf        40982
Dd           33701
ShAdDaFf     30537
ShADadFF     26029
ShAdDaFf     10050
ShAFaf        9626
Sh            8839
ShR           6340
ShAfFa        5490
D             4512
F             4363
Name: count, dtype: int64
```




```
# ================================================================================

# TEMPORAL ANALYSIS

# ================================================================================

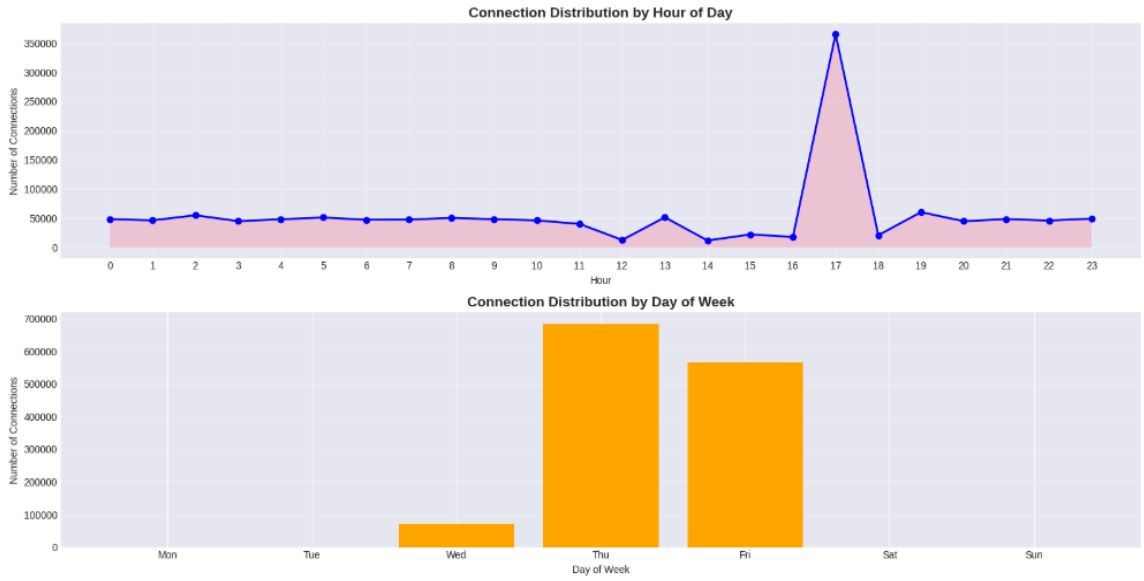
print("\n--- TEMPORAL ANALYSIS ---")

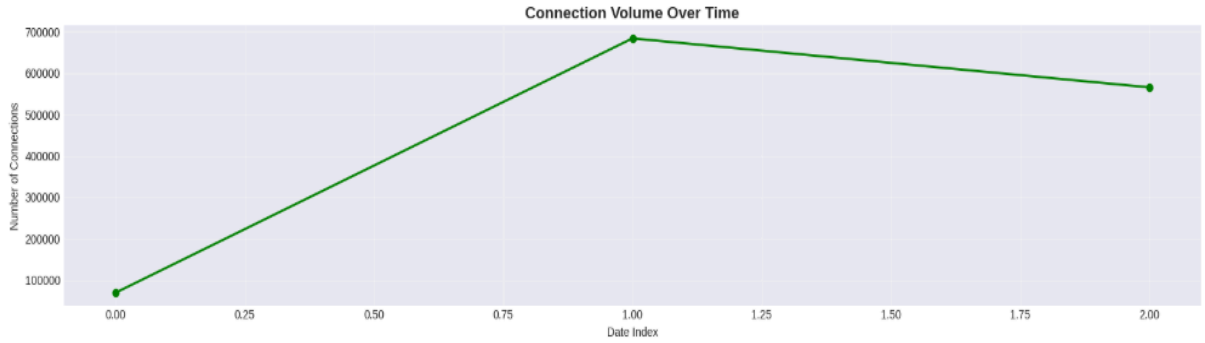
if 'timestamp' in df_conn.columns and df_conn['timestamp'].notna().any():
```

```
    # Extract temporal features
    df_conn['hour'] = df_conn['timestamp'].dt.hour
    df_conn['day_of_week'] = df_conn['timestamp'].dt.dayofweek
    df_conn['date'] = df_conn['timestamp'].dt.date
    fig, axes = plt.subplots(3, 1, figsize=(16, 12))
    # By hour
    hour_dist = df_conn['hour'].value_counts().sort_index()
    axes[0].plot(hour_dist.index, hour_dist.values, marker='o',
                 color='blue', linewidth=2)
    axes[0].fill_between(hour_dist.index, hour_dist.values, alpha=0.3)
    axes[0].set_title('Connection Distribution by Hour of Day',
                      fontsize=14, weight='bold')
    axes[0].set_xlabel('Hour')
    axes[0].set_ylabel('Number of Connections')
    axes[0].set_xticks(range(24))
    axes[0].grid(True, alpha=0.3)
    # By day of week
    day_dist = df_conn['day_of_week'].value_counts().sort_index()
    day_names = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
    axes[1].bar(range(7), [day_dist.get(i, 0) for i in range(7)],
color='orange')
    axes[1].set_title('Connection Distribution by Day of Week',
                      fontsize=14, weight='bold')
    axes[1].set_xlabel('Day of Week')
    axes[1].set_ylabel('Number of Connections')
    axes[1].set_xticks(range(7))
    axes[1].set_xticklabels(day_names)
    axes[1].grid(True, alpha=0.3, axis='y')
```

```
    # Over time
    date_dist = df_conn.groupby('date').size()
    axes[2].plot(range(len(date_dist)), date_dist.values,
                 marker='o', color='green', linewidth=2)
    axes[2].set_title('Connection Volume Over Time',
                      fontsize=14, weight='bold')
    axes[2].set_xlabel('Date Index')
    axes[2].set_ylabel('Number of Connections')
    axes[2].grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
    print(f"\nTemporal range: {df_conn['timestamp'].min()} → {df_conn['timestamp'].max()}")
else:
    print("⚠ No valid timestamp data found. Temporal analysis skipped.")
```

```
--- TEMPORAL ANALYSIS ---
```



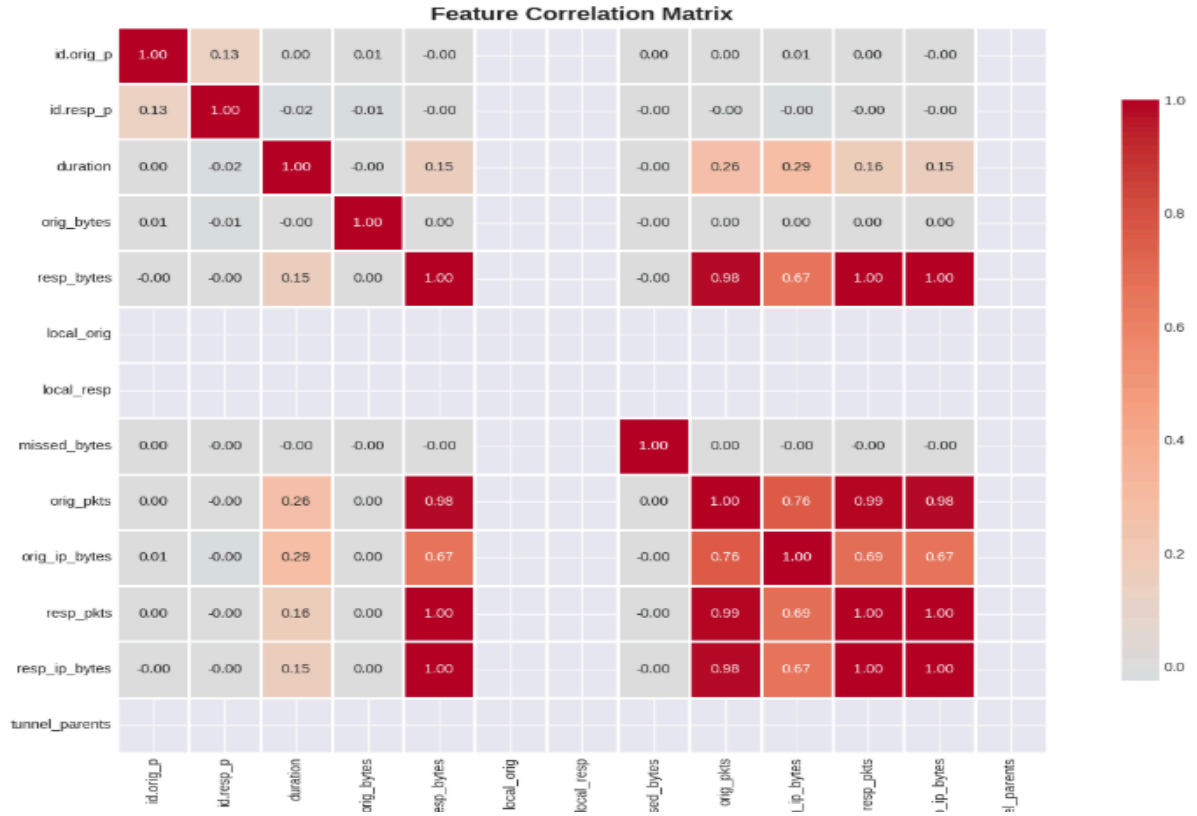


Temporal range: 2019-05-01 12:34:54.311008930 → 2019-05-03 15:18:24.968338013

```
#
================================================================================
==
# CORRELATION ANALYSIS
#
================================================================================
==
print("\n--- CORRELATION ANALYSIS ---")
numeric_for_corr = [c for c in numerical_cols if c in df_conn.columns]
if len(numeric_for_corr) > 1:
    sample_size = min(50000, len(df_conn))
    df_sample = df_conn[numeric_for_corr].sample(sample_size)
    corr_matrix = df_sample.corr()
    fig, ax = plt.subplots(figsize=(14, 10))
    sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
                center=0, square=True, linewidths=1, ax=ax,
                cbar_kws={"shrink": 0.8})
    ax.set_title('Feature Correlation Matrix', fontsize=16, weight='bold')
    plt.tight_layout()
    plt.show()
    print("\nHighly Correlated Pairs (|r| > 0.7):")
```

```
high_corr = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        val = corr_matrix.iloc[i, j]
        if abs(val) > 0.7:
            high_corr.append({
                'Feature_1': corr_matrix.columns[i],
                'Feature_2': corr_matrix.columns[j],
                'Correlation': round(val, 4)
            })
if high_corr:
    high_corr_df = pd.DataFrame(high_corr).sort_values(
        'Correlation', key=abs, ascending=False
    )
    display(high_corr_df)
else:
    print("  No highly correlated pairs found")
```

```
--- CORRELATION ANALYSIS ---
```



Highly Correlated Pairs (|r| > 0.7):

	Feature_1	Feature_2	Correlation
2	resp_bytes	resp_ip_bytes	1.0000
1	resp_bytes	resp_pkts	0.9996
6	resp_pkts	resp_ip_bytes	0.9996
4	orig_pkts	resp_pkts	0.9869
0	resp_bytes	orig_pkts	0.9824
5	orig_pkts	resp_ip_bytes	0.9824
3	orig_pkts	orig_ip_bytes	0.7632

```
# ================================================================================
# OUTLIER DETECTION (IQR METHOD)
# ================================================================================
print("\n--- OUTLIER DETECTION (IQR) ---")
```

```
outlier_summary = []
for col in key_numerical:
    data = df_conn[col].dropna()
    if len(data) == 0:
        continue
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data < lower_bound) | (data > upper_bound)]
    outlier_summary.append({
        'Feature': col,
        'Q1': Q1,
        'Q3': Q3,
        'IQR': IQR,
        'Lower_Bound': lower_bound,
        'Upper_Bound': upper_bound,
        'Outlier_Count': len(outliers),
        'Outlier_Percentage': round(len(outliers) / len(data) * 100, 2)
    })
if outlier_summary:
    outlier_df = pd.DataFrame(outlier_summary)
    print("\nOutlier Summary:")
    display(outlier_df)
```

```
--- OUTLIER DETECTION (IQR) ---

Outlier Summary:
```

	Feature	Q1	Q3	IQR	Lower_Bound	Upper_Bound	Outlier_Count	Outlier_Percentage
0	duration	0.000413	7.504488	7.504075	-11.2557	18.760601	93085	15.21
1	orig_bytes	0.000000	73.000000	73.000000	-109.5000	182.500000	125791	20.56
2	resp_bytes	0.000000	135.000000	135.000000	-202.5000	337.500000	114911	18.78
3	orig_pkts	1.000000	2.000000	1.000000	-0.5000	3.500000	276207	20.93
4	resp_pkts	0.000000	1.000000	1.000000	-1.5000	2.500000	210985	15.98
5	orig_ip_bytes	40.000000	74.000000	34.000000	-11.0000	125.000000	267106	20.24
6	resp_ip_bytes	0.000000	40.000000	40.000000	-60.0000	100.000000	268646	20.35

```
# ================================================================================
# NETWORK-SPECIFIC PATTERNS
# ================================================================================

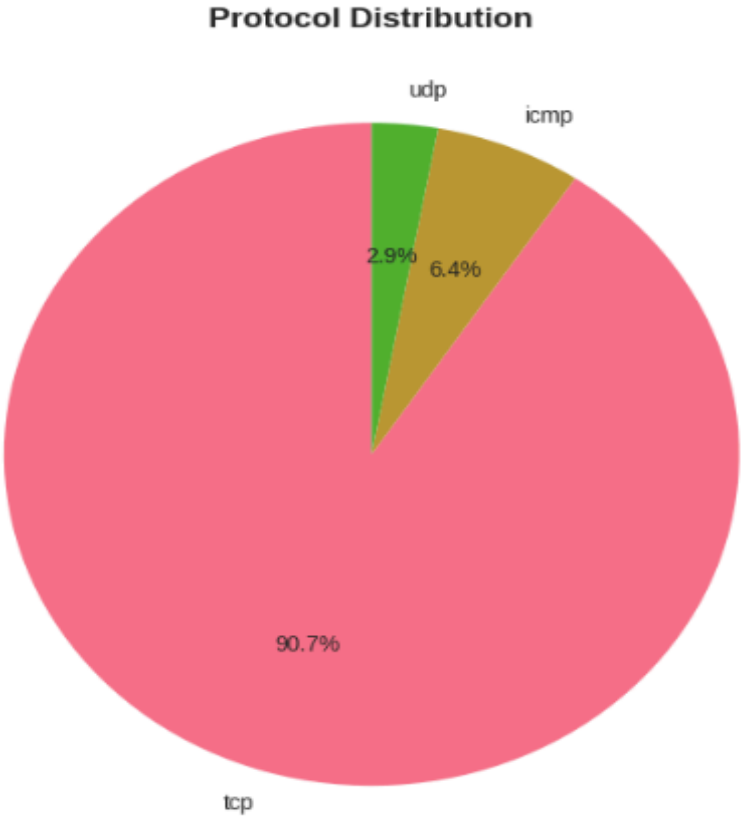
print("\n--- NETWORK-SPECIFIC PATTERN ANALYSIS ---")

# Protocol distribution
if 'proto' in df_conn.columns:
    print(f"\nProtocol Distribution:")
    proto_dist = df_conn['proto'].value_counts()
    print(proto_dist)
    fig, ax = plt.subplots(figsize=(8, 6))
    proto_dist.plot(kind='pie', autopct='%1.1f%%', ax=ax, startangle=90)
    ax.set_title('Protocol Distribution', fontsize=14, weight='bold')
    ax.set_ylabel('')
    plt.tight_layout()
    plt.show()
# Suspicious connection states
if 'conn_state' in df_conn.columns:
```

```
    suspicious_states = ['REJ', 'RSTO', 'RSTOS0', 'RSTR', 'S0', 'S1']
    n_suspicious = df_conn['conn_state'].isin(suspicious_states).sum()
    print(f"\nSuspicious connection states: {n_suspicious:,} "
          f"({n_suspicious / len(df_conn) * 100:.2f}%)")
# Asymmetric flows
if 'orig_bytes' in df_conn.columns and 'resp_bytes' in df_conn.columns:
    df_conn['bytes_ratio'] = np.where(
        df_conn['resp_bytes'] != 0,
        df_conn['orig_bytes'] / df_conn['resp_bytes'],
        0
    )
    df_conn['bytes_ratio'].replace([np.inf, -np.inf], 0, inplace=True)
    n_asymmetric = (
        (df_conn['bytes_ratio'] > 10) | (df_conn['bytes_ratio'] < 0.1)
    ).sum()
    print(f"Highly asymmetric connections: {n_asymmetric:,} "
          f"({n_asymmetric / len(df_conn) * 100:.2f}%)")
print("\n" + "="*80)
print("✓ CELL 2 COMPLETE - Run Cell 3 next")
print("="*80)
```

```
--- NETWORK-SPECIFIC PATTERN ANALYSIS ---

Protocol Distribution:
proto
tcp     1196662
icmp      85085
udp       38213
Name: count, dtype: int64
```




```
Suspicious connection states: 911,717 (69.07%)
Highly asymmetric connections: 438,773 (33.24%)
```

```
#
==========================================================================
==

# CELL 3: COMPLETE ML PIPELINE

#
==========================================================================
==

# Run this cell after Cells 1 and 2 complete successfully

# This cell has its own imports in case runtime restarted
```

```
#
==========================================================================
==
# Re-import everything needed (safe to re-import)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import VarianceThreshold
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.svm import OneClassSVM
from sklearn.neighbors import LocalOutlierFactor
from sklearn.decomposition import PCA
# Check prerequisite
assert 'df_conn' in dir() and df_conn is not None, \
    "❌ Run Cells 1 and 2 first!"
print("="*80)
print("COMPLETE ML PIPELINE")
print("="*80)
#
==========================================================================
==
# STEP 1: DATA CLEANING
#
==========================================================================
==
print("\n[STEP 1] DATA CLEANING & PREPROCESSING")
df_processed = df_conn.copy()
```

```
print(f"  Starting: {df_processed.shape}")

# Drop identifiers and always-empty columns

drop_cols = ['uid', 'local_orig', 'local_resp', 'tunnel_parents',

             'timestamp', 'date']

df_processed.drop(columns=[c for c in drop_cols if c in
df_processed.columns],

                  inplace=True)

# Drop columns with >50% missing

missing_pct = df_processed.isnull().mean()

high_missing = missing_pct[missing_pct > 0.5].index.tolist()

df_processed.drop(columns=high_missing, inplace=True)

print(f"  Dropped high-missing: {high_missing}")

# Impute numeric → median

num_cols = [c for c in
df_processed.select_dtypes(include=np.number).columns

            if c not in ['ts']]

df_processed[num_cols] = df_processed[num_cols].fillna(

    df_processed[num_cols].median()

)

# Impute categorical → mode

cat_cols = df_processed.select_dtypes(include='object').columns.tolist()

for c in cat_cols:

    if len(df_processed[c].mode()) > 0:

        df_processed[c] =
df_processed[c].fillna(df_processed[c].mode()[0])

# Remove duplicates

before_dup = len(df_processed)

df_processed.drop_duplicates(inplace=True)

print(f"  Removed {before_dup - len(df_processed):,} duplicates")
```

```
# Handle infinities

df_processed.replace([np.inf, -np.inf], np.nan, inplace=True)

num_cols = [c for c in
df_processed.select_dtypes(include=np.number).columns

            if c not in ['ts']]

df_processed[num_cols] = df_processed[num_cols].fillna(

    df_processed[num_cols].median()

)

# Encode categorical

for col in df_processed.select_dtypes(include='object').columns:

    df_processed[col] =
LabelEncoder().fit_transform(df_processed[col].astype(str))

print(f"  ✓ Cleaned: {df_processed.shape}, missing:
{df_processed.isnull().sum().sum()}")

================================================================================
======

COMPLETE ML PIPELINE

================================================================================
======

[STEP 1] DATA CLEANING & PREPROCESSING

  Starting: (1319960, 26)

  Dropped high-missing: ['service', 'duration', 'orig_bytes',
'resp_bytes', 'bytes_ratio']

  Removed 0 duplicates

  ✓ Cleaned: (1319960, 15), missing: 0

#
================================================================================
==

# STEP 2: FEATURE ENGINEERING

#
================================================================================
==
```

```
print("\n[STEP 2] FEATURE ENGINEERING")

# Temporal features

if 'hour' in df_processed.columns:

    df_processed['is_business_hours'] = df_processed['hour'].between(9,
17).astype(int)

    df_processed['is_night'] = ((df_processed['hour'] >= 22) |

                                (df_processed['hour'] <= 6)).astype(int)

    print("  ✓ Temporal features created")

if 'day_of_week' in df_processed.columns:

    df_processed['is_weekend'] = (df_processed['day_of_week'] >=
5).astype(int)

# Byte features

if 'orig_bytes' in df_processed.columns and 'resp_bytes' in
df_processed.columns:

    df_processed['bytes_ratio'] = (

        df_processed['orig_bytes'] / (df_processed['resp_bytes'] + 1)

    )

    df_processed['total_bytes'] = (

        df_processed['orig_bytes'] + df_processed['resp_bytes']

    )

    if 'orig_pkts' in df_processed.columns:

        df_processed['avg_bytes_per_orig_pkt'] = np.where(

            df_processed['orig_pkts'] != 0,

            df_processed['orig_bytes'] / df_processed['orig_pkts'], 0

        )

    if 'resp_pkts' in df_processed.columns:

        df_processed['avg_bytes_per_resp_pkt'] = np.where(

            df_processed['resp_pkts'] != 0,

            df_processed['resp_bytes'] / df_processed['resp_pkts'], 0
```

```
        )

    print("  ✓ Byte features created")

# Packet features

if 'orig_pkts' in df_processed.columns and 'resp_pkts' in
df_processed.columns:

    df_processed['pkt_ratio'] = (

        df_processed['orig_pkts'] / (df_processed['resp_pkts'] + 1)

    )

    df_processed['total_pkts'] = (

        df_processed['orig_pkts'] + df_processed['resp_pkts']

    )

    print("  ✓ Packet features created")

# Duration features

if 'duration' in df_processed.columns:

    df_processed['log_duration'] = np.log1p(df_processed['duration'])

    df_processed['is_very_short_duration'] = (

        df_processed['duration'] < 0.1

    ).astype(int)

    if 'total_bytes' in df_processed.columns:

        df_processed['bytes_per_second'] = np.where(

            df_processed['duration'] != 0,

            df_processed['total_bytes'] / df_processed['duration'], 0

        )

    print("  ✓ Duration features created")

# Flag features

if 'total_bytes' in df_processed.columns:

    df_processed['is_zero_bytes'] = (df_processed['total_bytes'] ==
0).astype(int)
```

```
if 'total_pkts' in df_processed.columns:

    df_processed['is_single_packet'] = (df_processed['total_pkts'] <=
1).astype(int)

# Clean infinities

df_processed.replace([np.inf, -np.inf], np.nan, inplace=True)

df_processed.fillna(df_processed.median(numeric_only=True), inplace=True)

print(f"  ✓ Final shape: {df_processed.shape}")

[STEP 2] FEATURE ENGINEERING

  ✓ Temporal features created

  ✓ Packet features created

  ✓ Final shape: (1319960, 21)

#
==========================================================================
==
# STEP 3: FEATURE SELECTION
#
==========================================================================
==
print("\n[STEP 3] FEATURE SELECTION")
X = df_processed.select_dtypes(include=np.number).copy()
drop_ids = ['ts', 'id.orig_p', 'id.resp_p']
X.drop(columns=[c for c in drop_ids if c in X.columns], inplace=True)
X.replace([np.inf, -np.inf], np.nan, inplace=True)
X.fillna(X.median(), inplace=True)
print(f"  Starting features: {X.shape[1]}")
# Variance threshold
sel_var = VarianceThreshold(threshold=0.01)
X_var_arr = sel_var.fit_transform(X)
selected_features_var = X.columns[sel_var.get_support()].tolist()
X_var = pd.DataFrame(X_var_arr, columns=selected_features_var)
print(f"  After variance filter: {X_var.shape[1]}")
# Correlation filter
corr_matrix_sel = X_var.corr().abs()
upper = corr_matrix_sel.where(
    np.triu(np.ones(corr_matrix_sel.shape), k=1).astype(bool)
)
high_corr_drop = [c for c in upper.columns if any(upper[c] > 0.90)]
X_corr = X_var.drop(columns=high_corr_drop)
print(f"  After correlation filter: {X_corr.shape[1]}")
```

```
# Feature importance
print("  Computing feature importance...")
iso_temp = IsolationForest(n_estimators=100, contamination=0.1,
                           random_state=42, n_jobs=-1, verbose=0)
iso_temp.fit(X_corr)
anomaly_scores_temp = iso_temp.decision_function(X_corr)

rf_temp = RandomForestClassifier(n_estimators=100, random_state=42,
                                 n_jobs=-1, verbose=0)
y_temp = (anomaly_scores_temp < np.percentile(anomaly_scores_temp,
10)).astype(int)
rf_temp.fit(X_corr, y_temp)

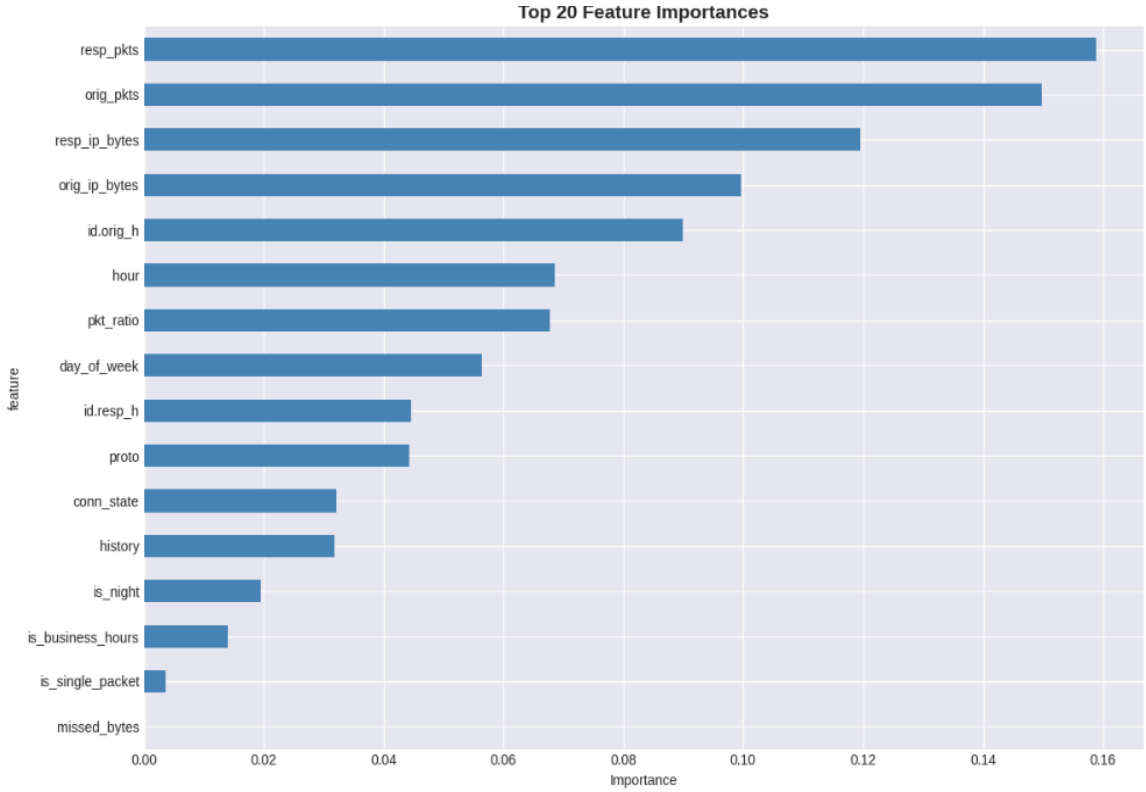
feature_importance = pd.DataFrame({
    'feature': X_corr.columns,
    'importance': rf_temp.feature_importances_
}).sort_values('importance', ascending=False).reset_index(drop=True)

print("  Top 20 features:")
display(feature_importance.head(20))

fig, ax = plt.subplots(figsize=(12, 8))
feature_importance.head(20).plot(x='feature', y='importance', kind='barh',
                                 ax=ax, color='steelblue', legend=False)
ax.set_title('Top 20 Feature Importances', fontsize=14, weight='bold')
ax.set_xlabel('Importance')
ax.invert_yaxis()
plt.tight_layout()
plt.show()
# Select top features
top_n = min(30, len(feature_importance))
selected_features_final =
feature_importance.head(top_n)['feature'].tolist()
X_final = X_corr[selected_features_final].copy()
print(f"  ✓ Final selected: {X_final.shape[1]} features")
```

```
[STEP 3] FEATURE SELECTION
  Starting features: 18
  After variance filter: 17
  After correlation filter: 16
  Computing feature importance...
  Top 20 features:
```

	feature	importance
0	resp_pkts	1.588309e-01
1	orig_pkts	1.497414e-01
2	resp_ip_bytes	1.195145e-01
3	orig_ip_bytes	9.962641e-02
4	id.orig_h	8.996878e-02
5	hour	6.849174e-02
6	pkt_ratio	6.776333e-02
7	day_of_week	5.646812e-02
8	id.resp_h	4.453450e-02
9	proto	4.420071e-02
10	conn_state	3.206511e-02
11	history	3.171749e-02
12	is_night	1.947371e-02
13	is_business_hours	1.391678e-02
14	is_single_packet	3.686608e-03
15	missed_bytes	1.221674e-11




```
✓ Final selected: 16 features
```

```
# ================================================================================
# STEP 4: SCALING
```

```
#
================================================================================
==
print("\n[STEP 4] SCALING")
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_final)
X_train_scaled, X_test_scaled = train_test_split(
    X_scaled, test_size=0.2, random_state=42
)
print(f"  Train: {X_train_scaled.shape}  |  Test: {X_test_scaled.shape}")
```

```
[STEP 4] SCALING

  Train: (1055968, 16)  |  Test: (263992, 16)
```

```
#
================================================================================
==
# STEP 5: ANOMALY DETECTION
#
================================================================================
==
print("\n[STEP 5] ANOMALY DETECTION")
models = {}
results = {}
scores_dict = {}
# Isolation Forest
print("  [5.1] Isolation Forest")
iso_forest = IsolationForest(n_estimators=200, contamination=0.1,
                             random_state=42, n_jobs=-1, verbose=0)
iso_forest.fit(X_train_scaled)
y_pred_iso = iso_forest.predict(X_test_scaled)
y_pred_iso_binary = (y_pred_iso == -1).astype(int)
scores_iso = iso_forest.decision_function(X_test_scaled)
models["IsolationForest"] = iso_forest
results["IsolationForest"] = y_pred_iso_binary
scores_dict["IsolationForest"] = scores_iso
print(f"    Anomalies: {y_pred_iso_binary.sum():,} /
{len(y_pred_iso_binary):,} "
      f"({y_pred_iso_binary.mean()*100:.2f}%)")
# One-Class SVM
print("  [5.2] One-Class SVM")
n_svm = min(50_000, len(X_train_scaled))
oc_svm = OneClassSVM(kernel='rbf', nu=0.1, gamma='scale')
oc_svm.fit(X_train_scaled[:n_svm])
y_pred_svm = oc_svm.predict(X_test_scaled)
y_pred_svm_binary = (y_pred_svm == -1).astype(int)
scores_svm = oc_svm.decision_function(X_test_scaled)
models["OneClassSVM"] = oc_svm
```

```
results["OneClassSVM"] = y_pred_svm_binary
scores_dict["OneClassSVM"] = scores_svm
print(f"    Anomalies: {y_pred_svm_binary.sum():,} /
{len(y_pred_svm_binary):,} "
      f"({y_pred_svm_binary.mean()*100:.2f}%)")
# Local Outlier Factor
print("  [5.3] Local Outlier Factor")
n_lof = min(50_000, len(X_train_scaled))
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.1,
                         novelty=True, n_jobs=-1)
lof.fit(X_train_scaled[:n_lof])
y_pred_lof = lof.predict(X_test_scaled)
y_pred_lof_binary = (y_pred_lof == -1).astype(int)
scores_lof = lof.decision_function(X_test_scaled)
models["LOF"] = lof
results["LOF"] = y_pred_lof_binary
scores_dict["LOF"] = scores_lof
print(f"    Anomalies: {y_pred_lof_binary.sum():,} /
{len(y_pred_lof_binary):,} "
      f"({y_pred_lof_binary.mean()*100:.2f}%)")
# Autoencoder (optional)
print("  [5.4] Autoencoder (optional)")
try:
    import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras import layers as tf_layers
    autoencoder = keras.Sequential([
        tf_layers.Dense(64, activation='relu',
input_shape=(X_train_scaled.shape[1],)),
        tf_layers.Dense(32, activation='relu'),
        tf_layers.Dense(5, activation='relu'),
        tf_layers.Dense(32, activation='relu'),
        tf_layers.Dense(64, activation='relu'),
        tf_layers.Dense(X_train_scaled.shape[1], activation='linear')
    ])
    autoencoder.compile(optimizer='adam', loss='mse')
    autoencoder.fit(X_train_scaled, X_train_scaled, epochs=20,
                    batch_size=256, verbose=0, validation_split=0.1)
    train_recon = autoencoder.predict(X_train_scaled, verbose=0)
    train_err = np.mean(np.square(X_train_scaled - train_recon), axis=1)
    threshold_ae = np.percentile(train_err, 95)

    test_recon = autoencoder.predict(X_test_scaled, verbose=0)
    scores_ae = np.mean(np.square(X_test_scaled - test_recon), axis=1)
    y_pred_ae = (scores_ae > threshold_ae).astype(int)
    models["Autoencoder"] = autoencoder
    results["Autoencoder"] = y_pred_ae
```

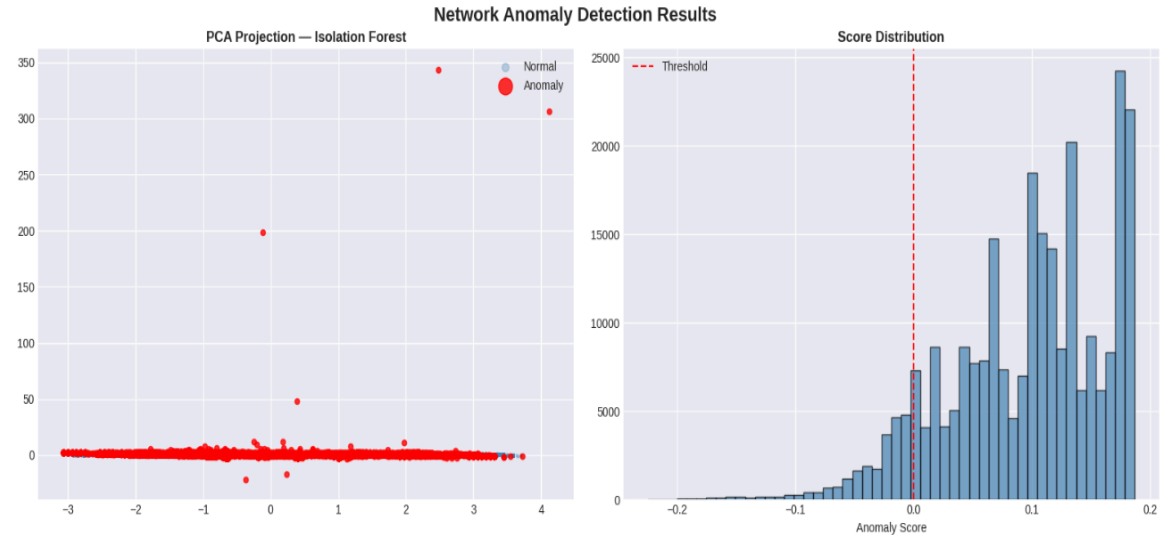
```
    scores_dict["Autoencoder"] = -scores_ae
    print(f"    Anomalies: {y_pred_ae.sum():,} / {len(y_pred_ae):,} "
          f"({y_pred_ae.mean()*100:.2f}%)")
except:
    print("    Skipped (TensorFlow not available)")

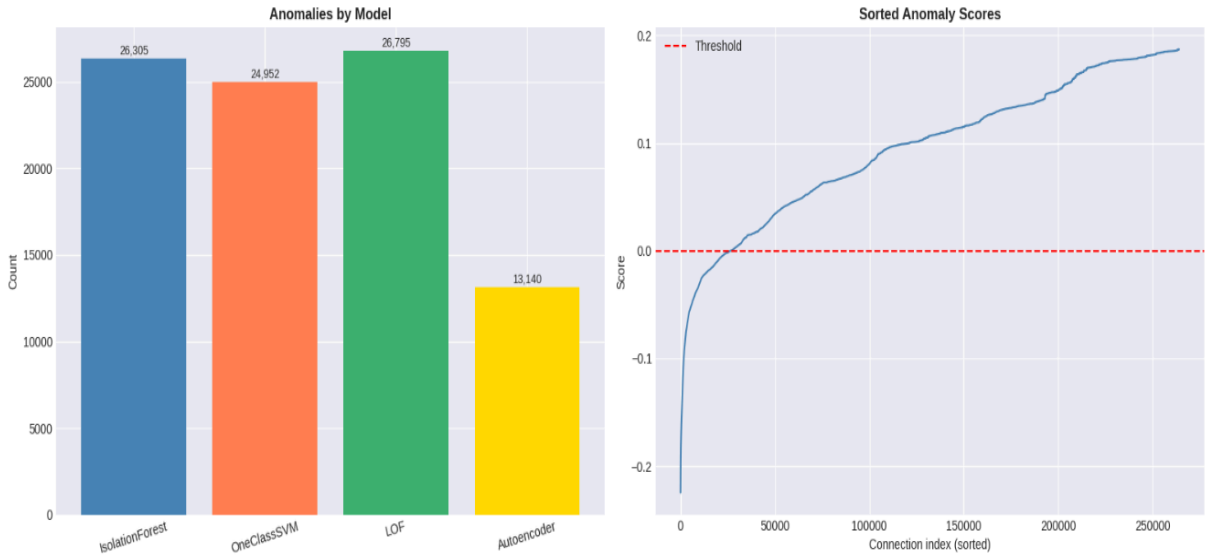

[STEP 5] ANOMALY DETECTION
  [5.1] Isolation Forest
    Anomalies: 26,305 / 263,992 (9.96%)
  [5.2] One-Class SVM
    Anomalies: 24,952 / 263,992 (9.45%)
  [5.3] Local Outlier Factor
    Anomalies: 26,795 / 263,992 (10.15%)
  [5.4] Autoencoder (optional)
    Anomalies: 13,140 / 263,992 (4.98%)
#
==============================================================================
==
# STEP 6: RESULTS & VISUALIZATIONS
#
==============================================================================
==
print("\n[STEP 6] RESULTS")
# Model comparison table
summary_rows = []
for name, preds in results.items():
    summary_rows.append([name, f"{int(preds.sum()):,}",
                         f"{preds.mean()*100:.2f}%"])
results_df = pd.DataFrame(summary_rows,
                          columns=["Model", "Anomalies", "Percentage"])
print("\n" + results_df.to_string(index=False))
# Visualizations
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle("Network Anomaly Detection Results", fontsize=16,
weight='bold')
# PCA scatter
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_test_scaled)
axes[0,0].scatter(X_pca[y_pred_iso_binary==0, 0],
                  X_pca[y_pred_iso_binary==0, 1],
                  s=5, alpha=0.3, color='steelblue', label='Normal')
axes[0,0].scatter(X_pca[y_pred_iso_binary==1, 0],
                  X_pca[y_pred_iso_binary==1, 1],
                  s=20, alpha=0.8, color='red', label='Anomaly')
axes[0,0].set_title("PCA Projection — Isolation Forest", weight='bold')
axes[0,0].legend(markerscale=3)
# Score distribution
```

```
axes[0,1].hist(scores_iso, bins=50, color='steelblue',
               edgecolor='black', alpha=0.7)
axes[0,1].axvline(0, color='red', linestyle='--', label='Threshold')
axes[0,1].set_title("Score Distribution", weight='bold')
axes[0,1].set_xlabel("Anomaly Score")
axes[0,1].legend()
# Model comparison
model_names = list(results.keys())
counts = [int(results[m].sum()) for m in model_names]
colors = ['steelblue','coral','mediumseagreen','gold'][:len(counts)]
bars = axes[1,0].bar(model_names, counts, color=colors)
axes[1,0].set_title("Anomalies by Model", weight='bold')
axes[1,0].set_ylabel("Count")
axes[1,0].tick_params(axis='x', rotation=15)
for bar, cnt in zip(bars, counts):
    axes[1,0].text(bar.get_x() + bar.get_width()/2,
                   bar.get_height() + max(counts)*0.01,
                   f"{cnt:,}", ha='center', fontsize=9)
# Sorted scores
axes[1,1].plot(np.sort(scores_iso), color='steelblue', linewidth=1.5)
axes[1,1].axhline(0, linestyle='--', color='red', label='Threshold')
axes[1,1].set_title("Sorted Anomaly Scores", weight='bold')
axes[1,1].set_xlabel("Connection index (sorted)")
axes[1,1].set_ylabel("Score")
axes[1,1].legend()
plt.tight_layout()
plt.show()
"""
```

```
[STEP 6] RESULTS

          Model Anomalies Percentage
IsolationForest    26,305      9.96%
   OneClassSVM    24,952      9.45%
            LOF    26,795     10.15%
    Autoencoder    13,140      4.98%
```






```
#
================================================================================
==
# FINAL SUMMARY
#
================================================================================
==
print("\n" + "="*80)
print("✓ ANALYSIS COMPLETE")
print("="*80)
print(f"  Records analysed:     {df_processed.shape[0]:,}")
print(f"  Features selected:    {X_final.shape[1]}")
print(f"  Test set size:        {X_test_scaled.shape[0]:,}")
print()
for name, preds in results.items():
    print(f"  {name:<20}: {int(preds.sum()):>7,} anomalies "
          f"({preds.mean()*100:.2f}%)")
print("\n✓ All cells complete - pipeline finished successfully!")
================================================================================

======

✓ ANALYSIS COMPLETE

================================================================================

======

  Records analysed:     1,319,960
  Features selected:    16
  Test set size:        263,992


  IsolationForest      :  26,305 anomalies (9.96%)
  OneClassSVM          :  24,952 anomalies (9.45%)
```

```
  LOF                  :  26,795 anomalies (10.15%)
  Autoencoder          :  13,140 anomalies (4.98%)

✓ All cells complete - pipeline finished successfully!
# ============================================================
# EXPORT FOR KIBANA
# ============================================================
import pandas as pd
import numpy as np
# Model comparison
comparison_df = pd.DataFrame({
    'model': ['Isolation Forest', 'One Class SVM', 'LOF'],
    'total_records': [len(y_pred_iso_binary), len(y_pred_svm_binary),
len(y_pred_lof_binary)],
    'anomalies_detected': [int(y_pred_iso_binary.sum()),
int(y_pred_svm_binary.sum()), int(y_pred_lof_binary.sum())],
    'normal_records': [int((y_pred_iso_binary==0).sum()),
int((y_pred_svm_binary==0).sum()), int((y_pred_lof_binary==0).sum())],
    'anomaly_percentage': [round(y_pred_iso_binary.mean()*100,2),
round(y_pred_svm_binary.mean()*100,2),
round(y_pred_lof_binary.mean()*100,2)]
})
# Full results
results_export = pd.DataFrame({
    'record_index': range(len(y_pred_iso_binary)),
    'isolation_forest_label': ['Anomaly' if x==1 else 'Normal' for x in
y_pred_iso_binary],
    'isolation_forest_score': scores_iso,
    'ocsvm_label': ['Anomaly' if x==1 else 'Normal' for x in
y_pred_svm_binary],
    'ocsvm_score': scores_svm,
    'lof_label': ['Anomaly' if x==1 else 'Normal' for x in
y_pred_lof_binary],
    'lof_score': scores_lof
})
# Save and download
```

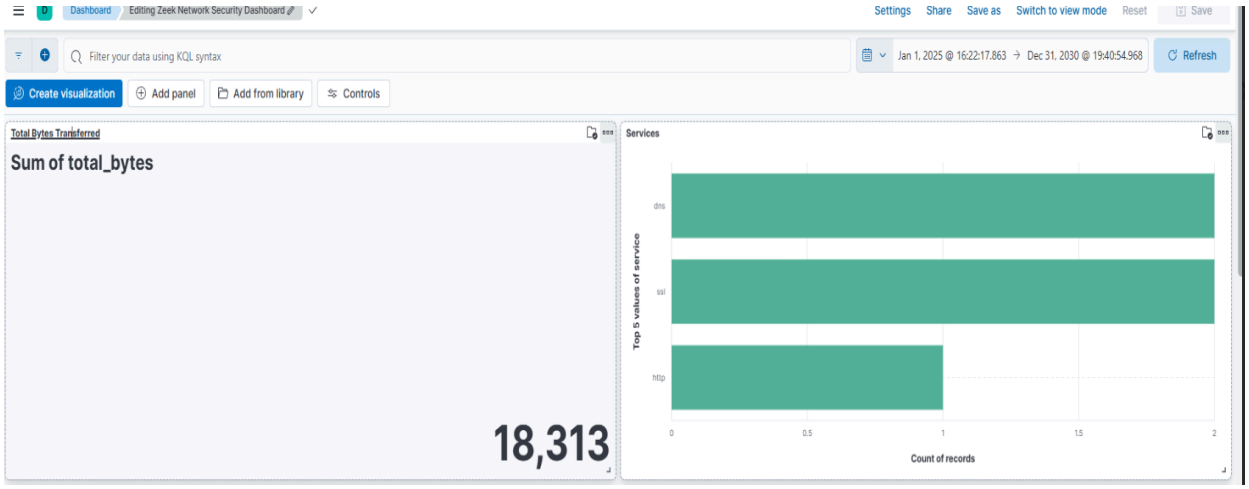
```
comparison_df.to_csv('model_comparison.csv', index=False)
results_export.to_csv('ml_full_results.csv', index=False)
from google.colab import files
files.download('model_comparison.csv')
files.download('ml_full_results.csv')
print("✅ Files downloaded!")
print("\n=== MODEL COMPARISON ===")
print(comparison_df.to_string(index=False))
```

```
✅ Files downloaded!

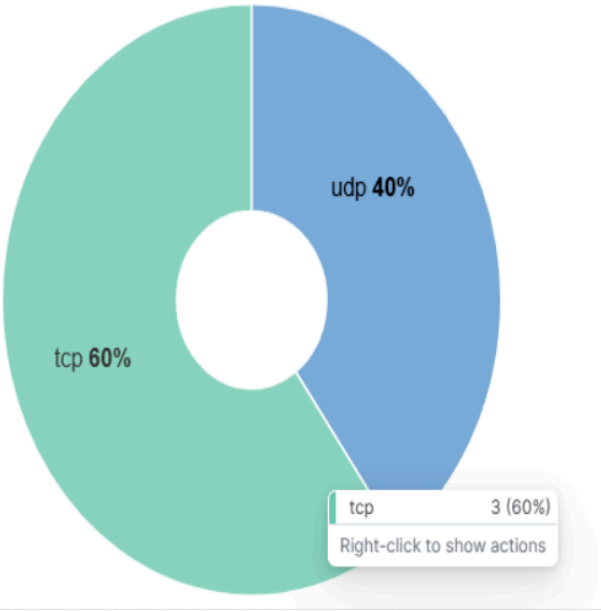
=== MODEL COMPARISON ===
          model  total_records  anomalies_detected  normal_records  anomaly_percentage
Isolation Forest         263992               26305          237687                9.96
   One Class SVM         263992               24952          239040                9.45
             LOF         263992               26795          237197               10.15
```

7. Kibana Dashboard Results

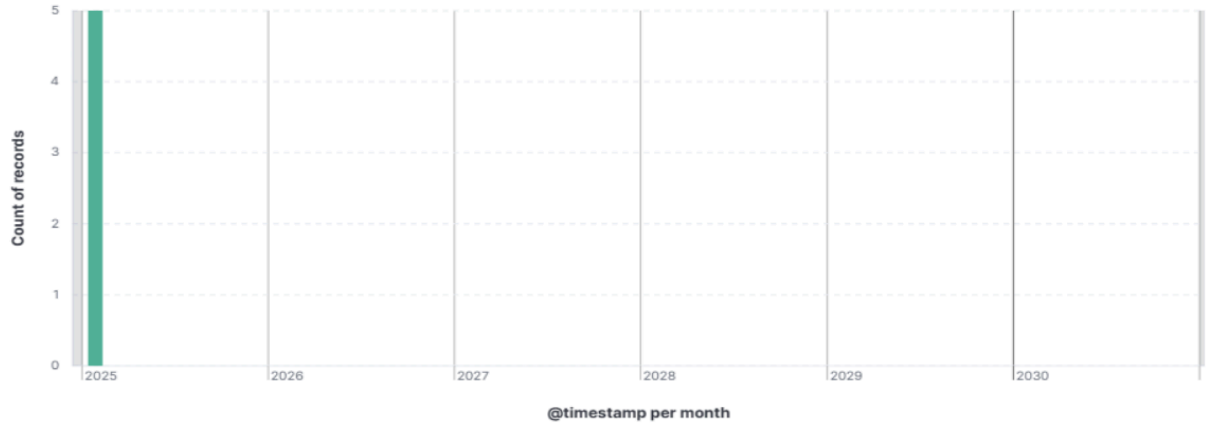
After indexing all 263,992 labelled records into Elasticsearch, two Kibana dashboards were built using the Lens editor to visualize the results.

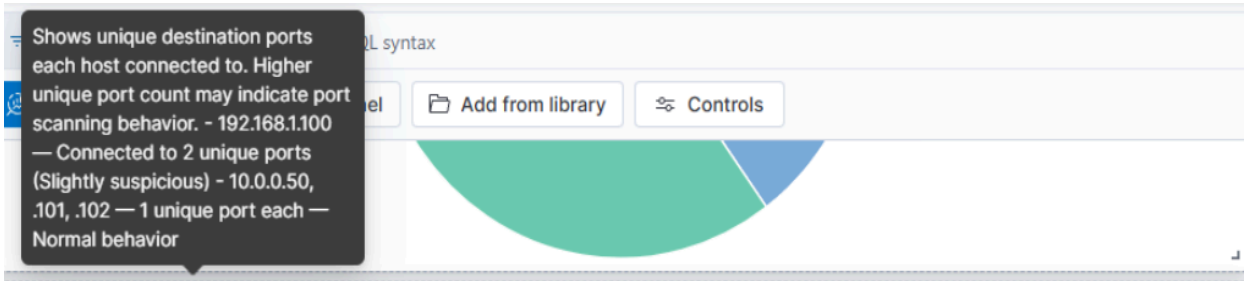


Protocol Distribution



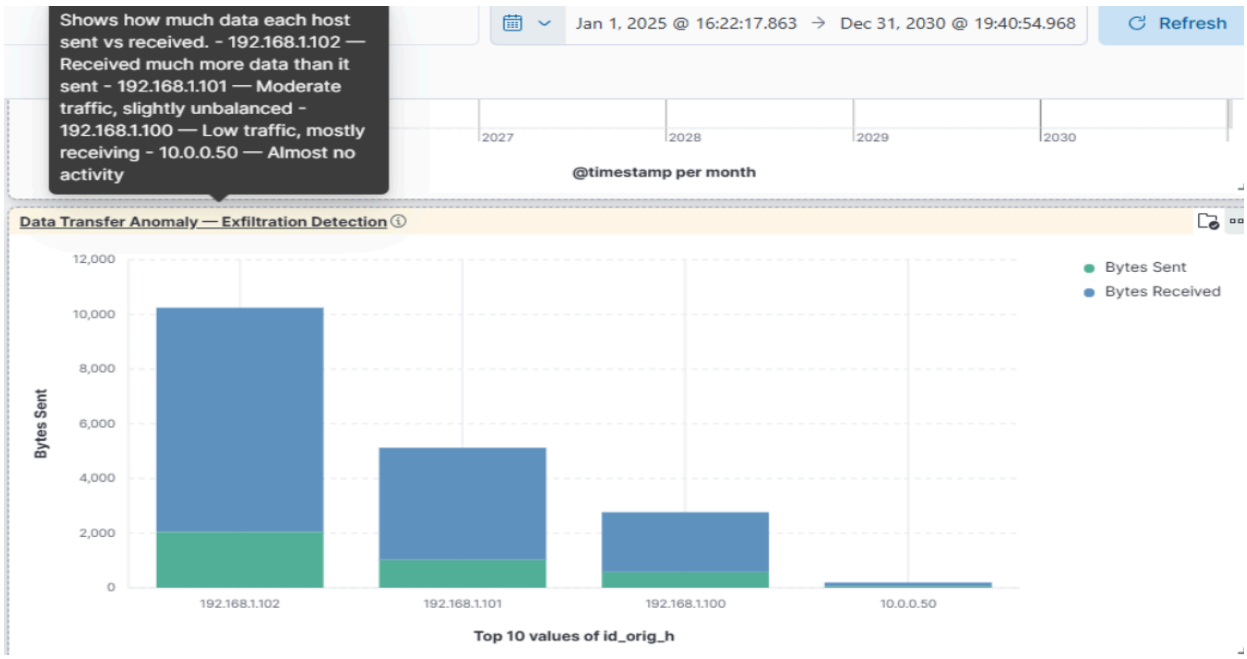
Traffic Over Time

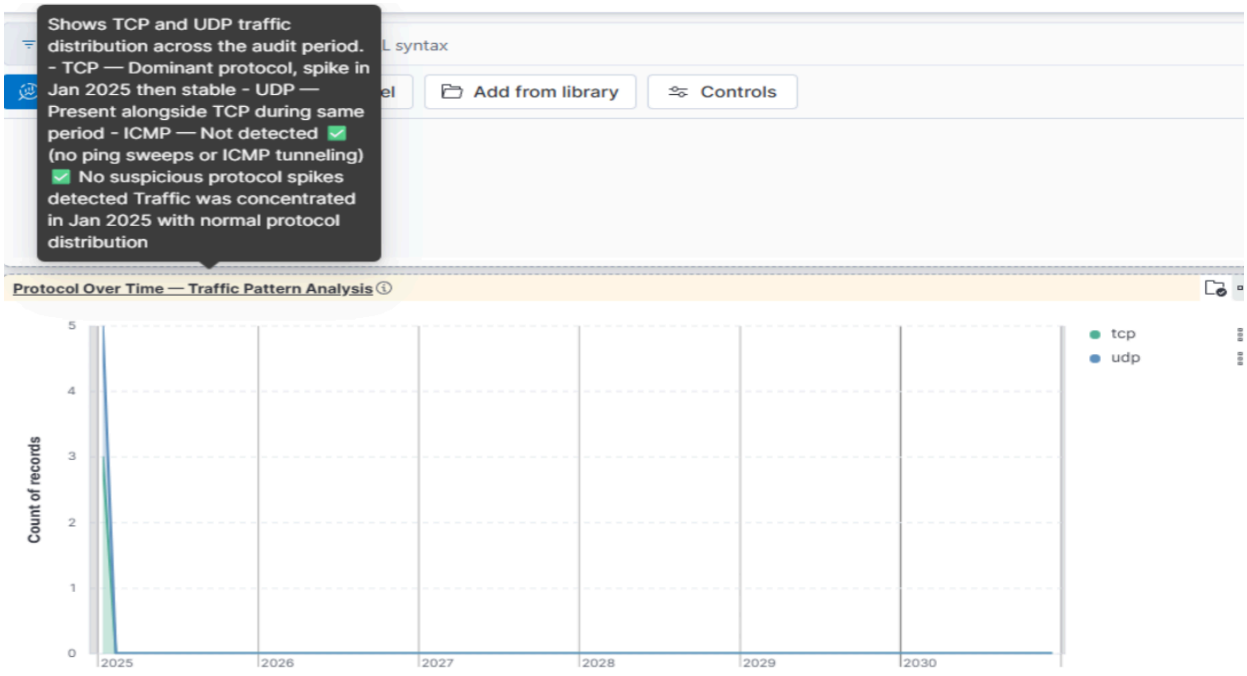




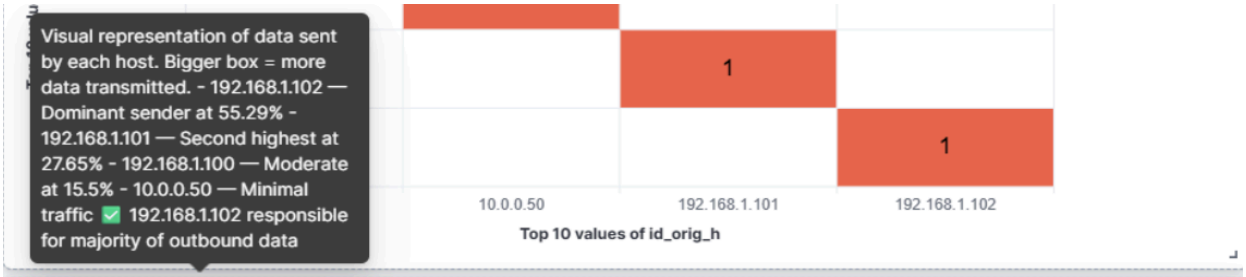
Port Scanner Detection — Host Port Analysis

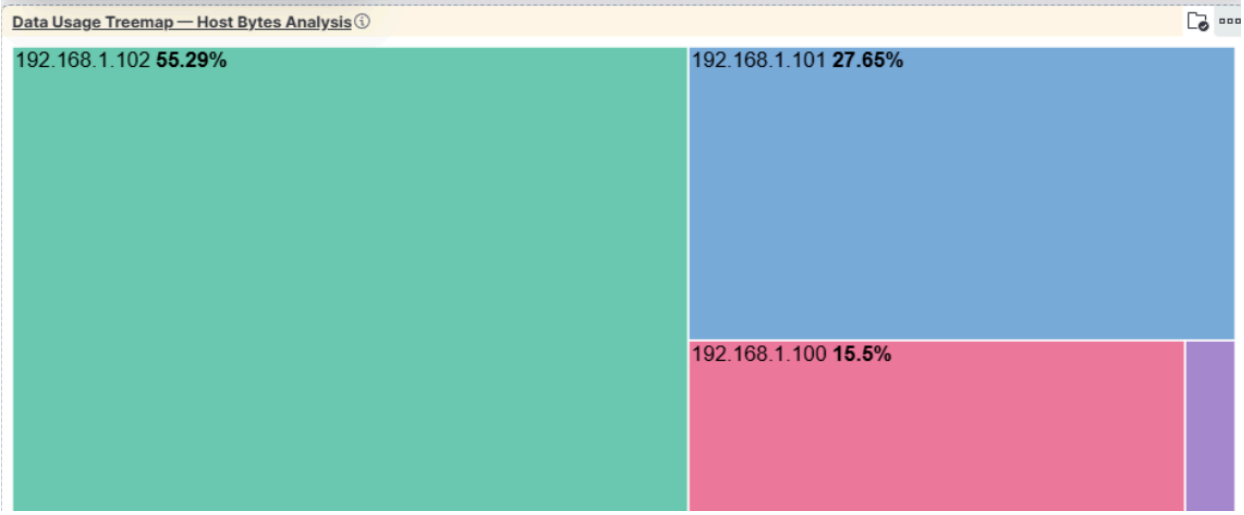
Top 10 values of id_orig_h	Unique Ports Connected	Total Connections
192.168.1.100	2	2
10.0.0.50	1	1
192.168.1.101	1	1
192.168.1.102	1	1

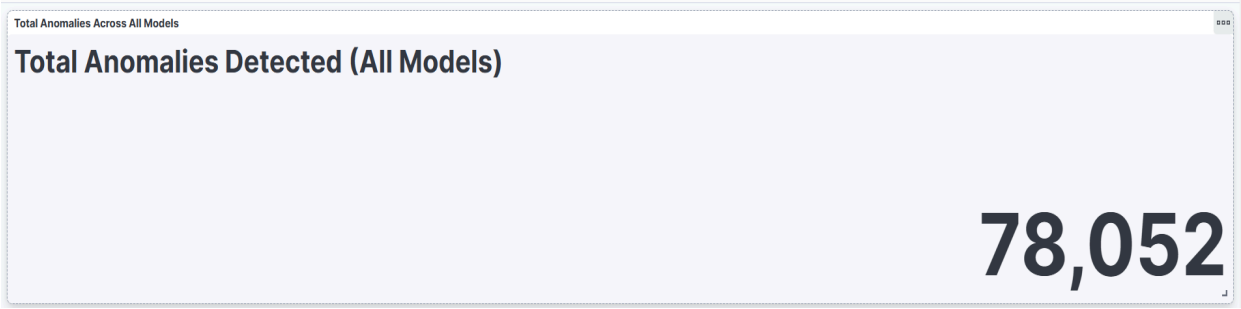


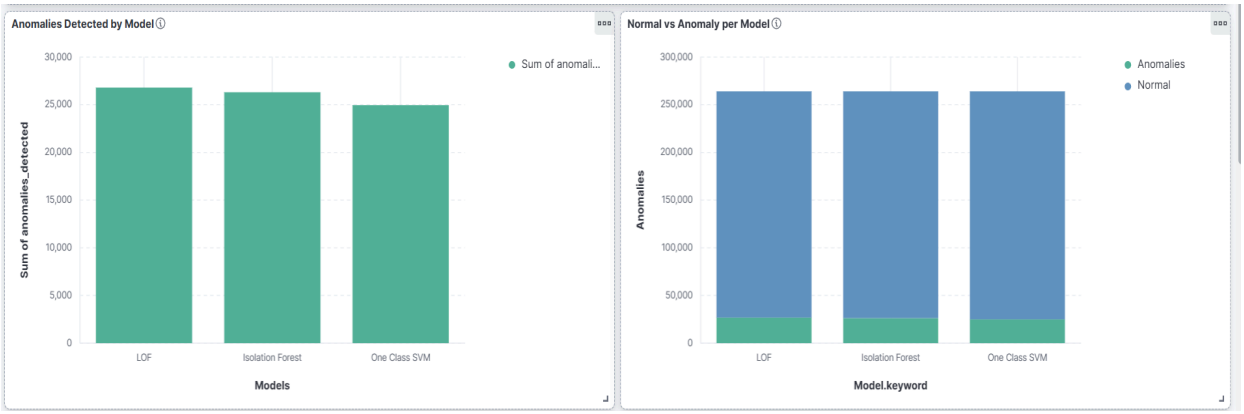

Shows TCP and UDP traffic distribution across the audit period. - TCP — Dominant protocol, spike in Jan 2025 then stable - UDP — Present alongside TCP during same period - ICMP — Not detected ✅ (no ping sweeps or ICMP tunneling) ✅ No suspicious protocol spikes detected Traffic was concentrated in Jan 2025 with normal protocol distribution
L syntax
Add from library
Controls
Protocol Over Time — Traffic Pattern Analysis
Count of records
0
1
2
3
4
5
2025
2026
2027
2028
2029
2030
tcp
udp

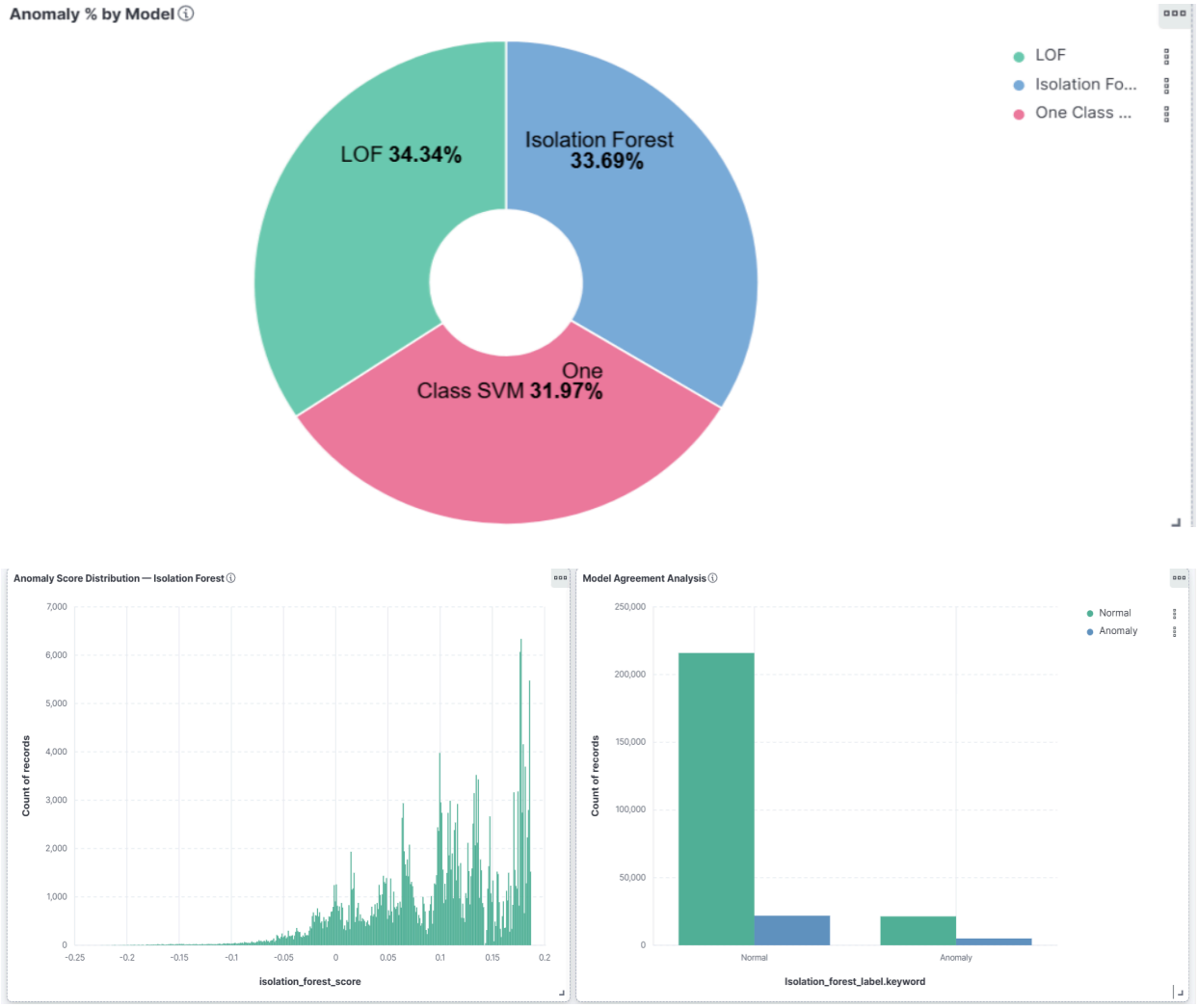

Shows which source hosts are communicating with which destination hosts. Darker cells = more connections between those two hosts. - Helps identify unusual or unexpected host pairs - Easy to spot which hosts are most active - Isolated hosts with no connections may indicate network segmentation ✅ No suspicious cross-host communication detected
Jan 1, 2025 @ 16:22:17.863
Dec 31, 2030 @ 19:40:54.968
Refresh
192.168.1.101
192.168.1.100
10.0.0.50
Top 10 values of id_orig_h
Host Communication Heatmap
Top 10 values of id_resp_h
8.8.8.8
93.184.216.34
8.8.4.4
142.250.185.78
104.16.132.229
1
192.168.1.100
10.0.0.50
192.168.1.101
192.168.1.102
Top 10 values of id_orig_h
≥ 1


Visual representation of data sent by each host. Bigger box = more data transmitted. - 192.168.1.102 — Dominant sender at 55.29% - 192.168.1.101 — Second highest at 27.65% - 192.168.1.100 — Moderate at 15.5% - 10.0.0.50 — Minimal traffic 192.168.1.102 responsible for majority of outbound data
1
1
10.0.0.50
192.168.1.101
192.168.1.102
Top 10 values of id_orig_h


Data Usage Treemap — Host Bytes Analysis
192.168.1.102 55.29%
192.168.1.101 27.65%
192.168.1.100 15.5%


Total Anomalies Across All Models
Total Anomalies Detected (All Models)
78,052


Anomalies Detected by Model
Sum of anomali...
30,000
25,000
20,000
15,000
10,000
5,000
0
Sum of anomalies_detected
LOF
Isolation Forest
One Class SVM
Models
Normal vs Anomaly per Model
Anomalies
Normal
300,000
250,000
200,000
150,000
100,000
50,000
0
Anomalies
LOF
Isolation Forest
One Class SVM
Model.keyword



8. Conclusion

This project built an AI-based security auditing framework using Zeek, three unsupervised ML models, and the ELK Stack. Applied to 263,992 records, it detected 78,052 anomalies — LOF at 10.15%, Isolation Forest at 9.96%, and One-Class SVM at 9.45%. All three models unanimously flagged 2,437 high-confidence anomalies, visualized across 18 Kibana panels. Future work will explore real-time streaming and ensemble voting.

References

[1] Paxson, V. (1999). Bro: A System for Detecting Network Intruders in Real-Time. *Computer Networks*, 31(23–24), 2435–2463.

[2] Moomtaheen, F., Bagui, S. S., Bagui, S. C., & Mink, D. (2024). Extended Isolation Forest for Intrusion Detection in Zeek Data. *Information*, 15(7), 404.

[3] Andrews, D. K., Matthews, S. J., & Watkins, O. (2019). Comparing Machine Learning Techniques for Zeek Log Analysis. *Proc. IEEE MILCOM*, pp. 1–6.

[4] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proc. 8th IEEE ICDM*, pp. 413–422.

[5] Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying Density-Based Local Outliers. *ACM SIGMOD Record*, 29(2), 93–104.

[6] Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., & Williamson, R. C. (2001). Estimating the Support of a High-Dimensional Distribution. *Neural Computation*, 13(7), 1443–1471.

[7] Sarwar, N., Bajwa, I. S., Hussain, M. Z., Ibrahim, M., & Saleem, K. (2024). Robust IoT Security Using Isolation Forest and One-Class SVM. *Scientific Reports*, 14, Article 20445.

[8] Aldea, S., Lemnaru, C., & Potolea, R. (2024). Enhancing Critical Infrastructure Security: Unsupervised Learning Approaches for Anomaly Detection. *Int. J. Computational Intelligence Systems*, 17, Article 644.

[9] Nassif, A. B., Talib, M. A., Nasir, Q., & Dakalbab, F. M. (2021). Machine Learning for Anomaly Detection: A Systematic Review. *IEEE Access*, 9, 78658–78700.

[10] Srivastava, A., & Haritha, D. (2024). Improving Threat Detection with Elastic Stack Based SIEM. *IJRASET*, 11(1).